

Chapter 53

Two-Wire Automotive Interface (TWAI)

The Two-wire Automotive Interface (TWAI®) is a multi-master, multi-cast communication protocol with functions such as error detection and signaling and inbuilt message priorities and arbitration. The TWAI protocol is suited for automotive and industrial applications (see Section 53.2 for more details).

ESP32-P4 contains three TWAI controllers, TWAI 0, TWAI 1, and TWAI 2. Each controller can individually be connected to a TWAI bus via an external transceiver. The TWAI controllers have many advanced features, and can be utilized in a wide range of use cases, such as automotive products, industrial automation controls, building automation, etc.

53.1 Features

Each TWAI controller on ESP32-P4 supports the following features:

- Compatibility with ISO 11898-1 protocol (CAN Specification 2.0)
- Standard Frame Format (11-bit ID) and Extended Frame Format (29-bit ID)
- Bit rates from 1 Kbit/s to 1 Mbit/s
- Multiple modes of operation:
 - Normal
 - Listen-only (no influence on bus)
 - Self-test (no acknowledgment required during data transmission)
- 64-byte Receive FIFO
- Special transmissions:
 - Single-shot transmissions (does not automatically re-transmit upon error)
 - Self-reception (the TWAI controller transmits and receives messages simultaneously)
- Acceptance Filter (supports Single and Dual-filter modes)
- Error detection and handling:
 - Error counters
 - Configurable error warning limit
 - Error code capture
 - Arbitration lost capture
 - Automatic transceiver standby

53.2 Protocol Overview

53.2.1 TWAI Properties

The TWAI protocol connects two or more nodes in a bus network, and allows nodes to exchange messages with deterministic delay. A TWAI bus has the following properties:

Single Channel and Non-Return-to-Zero: The bus has only one transmission line for single-channel communication, which is half-duplex. Synchronization is also implemented in this channel, so extra channels (e.g., clock or enable) are not required. The bit stream of a TWAI message is encoded using the Non-Return-to-Zero (NRZ) method.

Bit Values: The single channel can either be in a dominant or recessive state, representing a logical 0 and a logical 1 respectively. A node transmitting data in a dominant state always overrides the other node transmitting data in a recessive state. The physical implementation on the bus is left to the application level to decide (e.g., differential pair or a single wire).

Bit Stuffing: Certain fields of TWAI messages are bit-stuffed. A transmitter that transmits five consecutive bits of the same value (e.g., dominant value or recessive value) should automatically insert a complementary bit. Likewise, a receiver that receives five consecutive bits of the same value should treat the next bit as a stuffed bit. Bit stuffing is applied to the following fields: SOF, arbitration field, control field, data field, and CRC sequence (see Section 53.2.2 for more details).

Multi-Cast: All nodes receive the same bits as they are connected to the same bus. Data is consistent across all nodes unless there is a bus error (see Section 53.2.3 for more details).

Multi-Master: Any node can initiate a transmission. If a transmission is already ongoing, a node will wait until the current transmission is over before initiating a new transmission.

Message Priority and Arbitration: If two or more nodes simultaneously initiate a transmission, the TWAI protocol ensures that one node will win arbitration of the bus. The arbitration field of the message transmitted by each node is used to determine which node will win the arbitration.

Error Detection and Signaling: Each node actively monitors the bus for errors, and signals the detected errors by transmitting an error frame.

Fault Confinement: Each node maintains a set of error counters that are incremented/decremented according to a set of rules. When the error counters surpass a certain threshold, the node will automatically eliminate itself from the network by switching itself off.

Configurable Bit Rate: The bit rate for a single TWAI bus is configurable. However, all nodes on the same bus must operate at the same bit rate.

Transmitters and Receivers: At any point in time, a TWAI node can either be a transmitter or a receiver.

- A node generating a message is a transmitter. The node remains a transmitter until the bus is idle or until the node loses arbitration. Please note that there could be multiple nodes that act as transmitters during arbitration.
- All nodes that are not transmitters can work as receivers.

53.2.2 TWAI Messages

TWAI nodes use messages to transmit data, and signal errors to other nodes when detecting errors on the bus. Messages are split into various frame types, and different frame types have different frame formats.

The TWAI protocol has the following frame types:

- Data frame
- Remote frame
- Error frame
- Overload frame
- Interframe space

The TWAI protocol supports the following frame formats:

- Standard Frame Format (SFF) that uses a 11-bit identifier
- Extended Frame Format (EFF) that uses a 29-bit identifier

53.2.2.1 Data Frames and Remote Frames

Data frames are used by nodes to send data to other nodes, and can have a payload of 0 to 8 data bytes.

Remote frames are used for nodes to request a data frame with the same identifier from other nodes, and thus they do not contain any data bytes. However, data frames and remote frames share many fields. Figure 53.2-1 illustrates the fields and sub-fields of different frames and formats.

Arbitration Field

When two or more nodes transmit data or remote frames simultaneously, the arbitration field is used to determine which node will win arbitration of the bus. In the arbitration field, if a node transmits a recessive bit while detecting a dominant bit, it indicates that another node has overridden its recessive bit. Therefore, the node transmitting the recessive bit has lost arbitration of the bus and should immediately switch to be a receiver.

The arbitration field primarily consists of a frame identifier that is transmitted from the most significant bit first. Given that a dominant bit represents a logical 0, and a recessive bit represents a logical 1:

- A frame with the smallest ID value always wins arbitration.
- Given the same ID and format, data frames always prevail over remote frames due to their **RTR** bits being dominant.
- Given the same first 11 bits of ID, a Standard Format Data Frame always prevails over an Extended Format Data Frame due to its **SRR** bits being recessive.

Control Field

The control field primarily consists of the Data Length Code (DLC) which indicates the number of payload data bytes for a data frame, or the number of requested data bytes for a remote frame. The DLC is transmitted from the most significant bit first.

Data Field

The data field contains the actual payload data bytes of a data frame. Remote frames do not contain any data field.

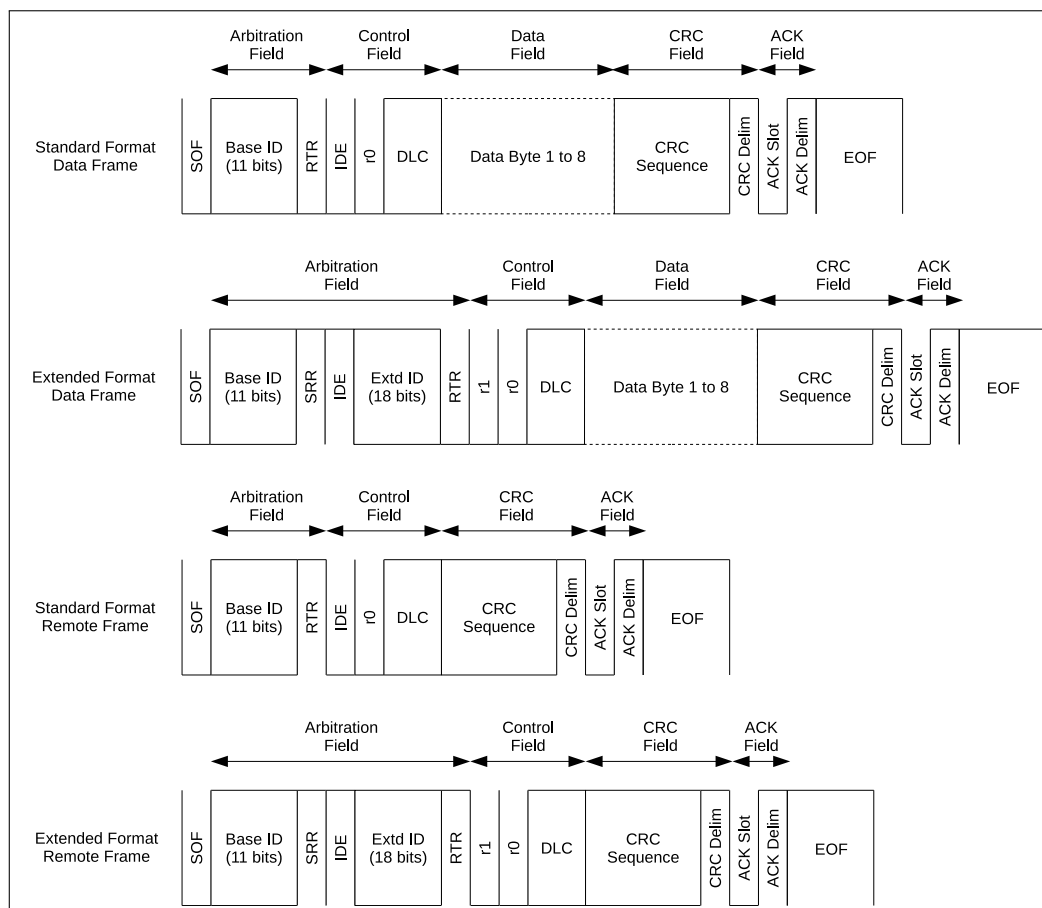


Figure 53.2-1. Bit Fields in Data Frames and Remote Frames

CRC Field

The CRC field primarily consists of a CRC sequence. The CRC sequence is a 15-bit cyclic redundancy code calculated from the de-stuffed contents (everything from the SOF to the end of the data field) of a data or remote frame.

ACK Field

The ACK field primarily consists of an ACK Slot and an ACK Delim. The ACK field indicates that the receiver has received an effective message from the transmitter.

Table 53.2-1. Data Frames and Remote Frames in SFF and EFF

Data/Remote Frames	Description
SOF	The SOF (Start of Frame) is a single dominant bit used to synchronize nodes on the bus.
Base ID	The Base ID (ID.28 to ID.18) is the 11-bit identifier for SFF, or the first 11 bits of the 29-bit identifier for EFF.
RTR	The RTR (Remote Transmission Request) bit indicates whether the message is a data frame (dominant) or a remote frame (recessive). This means that a remote frame will always lose arbitration to a data frame if they have the same ID.

Cont'd on next page

Table 53.2-1 – cont'd from previous page

Data/Remote Frames	Description
SRR	The SRR (Substitute Remote Request) bit is transmitted in EFF to substitute for the RTR bit at the same position in SFF.
IDE	The IDE (Identifier Extension) bit indicates whether the message is SFF (dominant) or EFF (recessive). This means that a SFF frame will always win arbitration over an EFF frame if they have the same Base ID.
Extd ID	The Extended ID (ID.17 to ID.0) is the remaining 18 bits of the 29-bit identifier for EFF.
r1	The r1 bit (reserved bit 1) is always dominant.
r0	The r0 bit (reserved bit 0) is always dominant.
DLC	The DLC (Data Length Code) is 4-bit long and should contain any value from 0 to 8. Data frames use the DLC to indicate the number of data bytes in the data frame. Remote frames used the DLC to indicate the number of data bytes to request from another node.
Data Bytes	The data payload of data frames. The number of bytes should match the value of DLC. Data byte 0 is transmitted first, and each data byte is transmitted from the most significant bit first.
CRC Sequence	The CRC sequence is a 15-bit cyclic redundancy code.
CRC Delim	The CRC Delim (CRC Delimiter) is a single recessive bit that follows the CRC sequence.
ACK Slot	The ACK Slot (Acknowledgment Slot) is intended for receiver nodes to indicate that the data or remote frame was received without any issue. The transmitter node will send a recessive bit in the ACK Slot and receiver nodes should override the ACK Slot with a dominant bit if the frame was received without errors.
ACK Delim	The ACK Delim (Acknowledgment Delimiter) is a single recessive bit.
EOF	The EOF (End of Frame) marks the end of a data or remote frame, and consists of seven recessive bits.

53.2.2.2 Error and Overload Frames

Error Frames

Error frames are transmitted when a node detects a bus error. Error frames notably include an Error Flag which is made up of six consecutive bits of the same value, thus violating the bit-stuffing rule. Therefore, when a particular node detects a bus error and transmits an error frame, all other nodes will then detect a stuff error and transmit their own error frames in response. This has the effect of propagating the detection of a bus error across all nodes on the bus.

When a node detects a bus error, it will transmit an error frame starting from the next bit. However, when a node detects a CRC error, the error frame will start at the bit following the ACK Delim (see Section 53.2.3 for more details). The following Figure 53.2-2 shows different fields of an error frame:

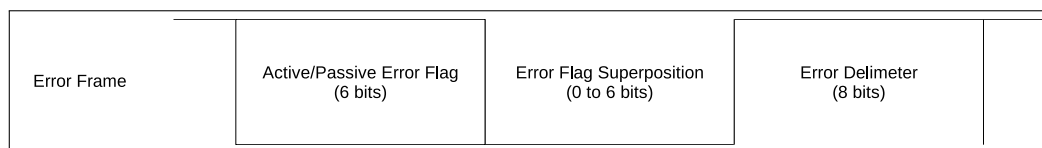


Figure 53.2-2. Fields of an Error Frame

Table 53.2-2. Error Frame

Error Frame	Description
Error Flag	The Error Flag has two forms, the Active Error Flag consisting of six dominant bits and the Passive Error Flag consisting of six recessive bits (unless overridden by dominant bits of other nodes). Active Error Flags are sent by error active nodes, whilst Passive Error Flags are sent by error passive nodes.
Error Flag Superposition	The Error Flag Superposition field meant to allow for other nodes on the bus to transmit their respective Active Error Flags. The superposition field can range from 0 to 6 bits, and ends when the first recessive bit is detected (i.e., the first bit of the Delimiter).
Error Delimiter	The Delimiter field marks the end of the error/overload frame, and consists of eight recessive bits.

Overload Frames

An overload frame has the same bit fields as an error frame containing an Active Error Flag. The key difference is in the cases that can trigger the transmission of an overload frame. Figure 53.2-3 below shows the bit fields of an overload frame.

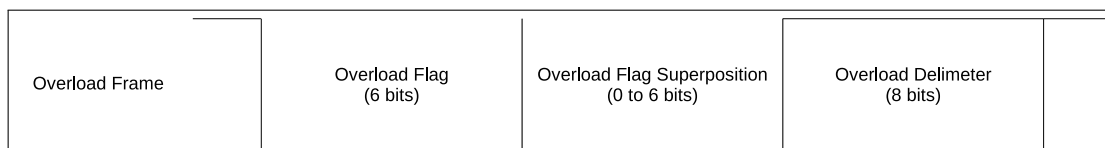


Figure 53.2-3. Fields of an Overload Frame

Table 53.2-3. Overload Frame

Overload Flag	Description
Overload Flag	Consists of six dominant bits. Same as an Active Error Flag.
Overload Flag Superposition	Allows for the superposition of Overload Flags from other nodes, similar to an Error Flag Superposition.
Overload Delimiter	Consists of eight recessive bits. Same as an Error Delimiter.

Overload frames will be transmitted in the following cases:

1. A receiver requires a delay of the next data or remote frame.
2. A dominant bit is detected at the first and second bit of [intermission](#).

3. A dominant bit is detected at the eighth (last) bit of an Error Delimiter. Note that in this case, TEC and REC will not be incremented (see Section 53.2.3 for more details).

Transmitting an overload frame due to one of the above cases must also satisfy the following rules:

- The start of an overload frame due to case 1 is only allowed to be started at the first bit time of an expected intermission. In this case, a maximum of two overload frames may be generated.
- The start of an overload frame due to case 2 and 3 is only allowed to be started one bit after detecting the dominant bit.

53.2.2.3 Interframe Space

The Interframe Space acts as a separator between frames. Before sending, data frames and remote frames must be separated from preceding frames by an Interframe Space, regardless of the preceding frame's type (data frame, remote frame, error frame, or overload frame). However, error frames and overload frames do not need to be separated from preceding frames.

Figure 53.2-4 shows the fields within an Interframe Space:

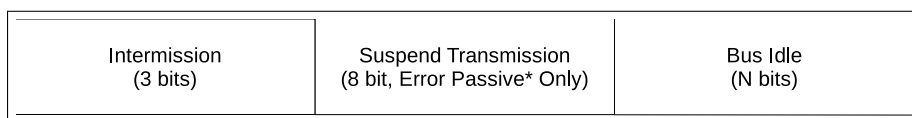


Figure 53.2-4. The Fields within an Interframe Space (*Error Passive)

Table 53.2-4. Interframe Space

Interframe Space	Description
Intermission	The Intermission consists of three recessive bits.
Suspend Transmission	An Error Passive node that has just transmitted a message must include a Suspend Transmission field. This field consists of eight recessive bits. Error Active nodes should not include this field.
Bus Idle	The Bus Idle field is of arbitrary length. Bus Idle ends when an SOF is transmitted. If a node has a pending transmission, the SOF should be transmitted at the first bit following Suspend Transmission.

53.2.3 TWAI Errors

53.2.3.1 Error Types

Bus Errors in TWAI are categorized into the following types:

Bit Error

A Bit Error occurs when a node transmits a bit value (i.e., dominant or recessive) but detects an opposite bit (e.g., a dominant bit is transmitted but a recessive is detected). However, if the transmitted bit is recessive and is located in the Arbitration Field, ACK Slot, or Passive Error Flag, then detecting a dominant bit will not be considered as a Bit Error.

Stuff Error

A stuff error occurs when six consecutive bits of the same value are detected (which violates the bit-stuffing encoding rules).

CRC Error

The receiver will calculate the CRC value based on the valid bits of received data frames and remote frames (before bit stuffing). A CRC Error occurs when the CRC calculated by the receiver does not match the CRC sequence in the received data frames and remote frames.

Format Error

A Format Error occurs when a format-fixed bit field of a message contains an illegal bit. For example, the r1 and r0 fields must be dominant.

ACK Error

An ACK Error occurs when a transmitter does not detect a dominant bit at the ACK Slot.

53.2.3.2 Error States

TWAI nodes implement fault confinement by maintaining two error counters in each node, where the counter values determine the error state. The two error counters are known as the Transmit Error Counter (TEC) and Receive Error Counter (REC). TWAI has the following error states:

Error Active

An Error Active node is able to participate in bus communication and transmit an Active Error Flag when it detects an error.

Error Passive

An Error Passive node is able to participate in bus communication and transmit a Passive Error Flag when it detects an error. Error Passive nodes that have transmitted data or remote frames must also include the Suspend Transmission field in the subsequent Interframe Space.

Bus Off

A Bus Off node is not permitted to influence the bus in any way (i.e., is not allowed to transmit data).

53.2.3.3 Error State Transition

1. A node becomes Error Passive when its TEC and/or REC is greater than or equal to 128. Though the node becomes Error Passive, it still sends an Active Error Flag. Note that once the REC has reached 128, any further increases to its value are invalid until the REC returns to a value less than 128.
2. A node becomes Bus Off when its TEC is greater than or equal to 256.
3. An Error Passive node becomes Error Active when both the TEC and REC are less than or equal to 127.
4. A Bus Off node can become Error Active (with both its TEC and REC reset to 0) after it monitors 128 occurrences of 11 consecutive recessive bits on the bus.

53.2.3.4 Error Counter Rules

The TEC and REC are incremented/decremented according to the following rules. **Note that more than one rule can apply to a given message transfer.**

TEC increment/decrement rules:

1. When a transmitter sends an Error Flag, the TEC is increased by 8. However, the following scenarios are exempt from this rule:
 - A transmitter is Error Passive and no dominant bit is detected when an Acknowledgment Error is detected and the Passive Error Flag is sent. In this case, the TEC should not be increased.
 - A transmitter transmits an Error Flag due to a Stuff Error during Arbitration. If the stuffed bit should have been recessive but was monitored as dominant, then the TEC should not be increased.
2. If a transmitter detects a Bit Error while sending an Active Error Flag or Overload Flag, the TEC is increased by 8.
3. A node can tolerate up to 7 consecutive dominant bits after sending an Active/Passive Error Flag, or Overload Flag. After detecting the 14th consecutive dominant bit when sending an Active Error Flag or Overload Flag, or the 8th consecutive dominant bit following a Passive Error Flag, a transmitter will increase its TEC by 8 and a receiver will increase its REC by 8. Every additional 8 consecutive dominant bits will also increase the TEC for transmitters or REC for receivers by 8 as well.
4. When a transmitter has transmitted a message, which means getting ACK and no errors until the EOF is completed, the TEC is decremented by 1, unless the TEC is already at 0.

REC increment/decrement rules:

1. When a receiver detects a dominant bit as the first bit after sending an Error Flag, the REC is increased by 8.
2. When a receiver detects an error, the REC is increased by 1, except when the detected error was a Bit Error during the transmission of an Active Error Flag or an Overload Flag.
3. If a receiver detects a Bit Error while sending an Active Error Flag or Overload Flag, the REC is increased by 8.
4. When a receiver successfully receives a message, which means getting no errors before ACK Slot and successfully sending ACK, the REC is decremented accordingly.
 - If the REC is between 1 and 127, it will be decremented by 1.
 - If the REC is greater than 127, it will be set to 127.
 - If the REC is 0, it will remain 0.

53.2.4 TWAI Bit Timing

53.2.4.1 Nominal Bit

The TWAI protocol allows a TWAI bus to operate at a particular bit rate. However, all nodes within a TWAI bus must operate at the same bit rate.

- **The Nominal Bit Rate** is defined as the number of bits transmitted per second.
- **The Nominal Bit Time** is defined as $1/\text{Nominal Bit Rate}$.

A single Nominal Bit Time is divided into multiple segments, and each segment is made up of multiple Time Quanta. A **Time Quantum** is a minimum unit of time, and is implemented as some form of prescaled clock signal in each node. Figure 53.2-5 illustrates the segments within a single Nominal Bit Time.

TWAI controllers will operate in time steps of one Time Quanta where the state of the TWAI bus is analyzed. If the bus states in two consecutive Time Quanta are different (i.e., recessive to dominant or vice versa), an edge is generated. The intersection of PBS1 and PBS2 is considered the Sample Point and the sampled bus value is considered the value of that bit.

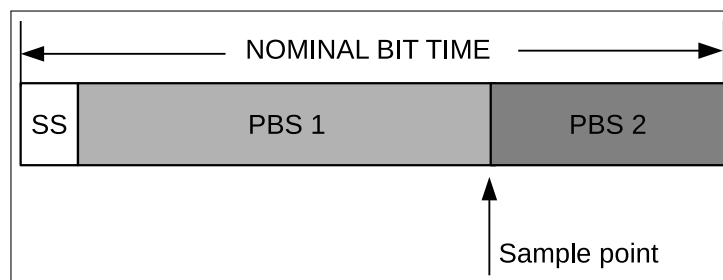


Figure 53.2-5. Layout of a Bit

Table 53.2-5. Segments of a Nominal Bit Time

Segment	Description
SS	The SS (Synchronization Segment) is 1 Time Quantum long. If all nodes are perfectly synchronized, the edge of a bit will lie in the SS.
PBS1	PBS1 (Phase Buffer Segment 1) can be 1 to 16 Time Quanta long. PBS1 is meant to compensate for the physical delay times within the network. PBS1 can also be lengthened for synchronization purposes.
PBS2	PBS2 (Phase Buffer Segment 2) can be 1 to 8 Time Quanta long. PBS2 is meant to compensate for the information processing time of nodes. PBS2 can also be shortened for synchronization purposes.

53.2.4.2 Hard Synchronization and Resynchronization

Due to clock skew and jitter, the bit timing of nodes on the same bus may become out of phase. Therefore, a bit edge may come before or after the SS. To ensure that the internal bit timing clocks of each node are kept in phase, TWAI has various methods of synchronization. The **Phase Error “e”** is measured in the number of Time Quanta and relative to the SS.

- A positive Phase Error ($e > 0$) is when the edge lies after the SS and before the Sample Point (i.e., the edge is late).
- A negative Phase Error ($e < 0$) is when the edge lies after the Sample Point of the previous bit and before SS (i.e., the edge is early).

To correct Phase Errors, there are two forms of synchronization, known as **Hard Synchronization** and **Resynchronization**. **Hard Synchronization** and **Resynchronization** obey the following rules:

- Only one synchronization may occur in a single bit time.
- Synchronizations only occur on recessive to dominant edges.

Hard Synchronization

Hard Synchronization occurs on the recessive to dominant (i.e., the first SOF bit after Bus Idle) edges when the bus is idle. All nodes will restart their internal bit timings so that the recessive to dominant edge lies within the SS of the restarted bit timing.

Resynchronization

Resynchronization occurs on recessive to dominant edges when the bus is not idle. If the edge has a positive Phase Error ($e > 0$), PBS1 is lengthened by a certain number of Time Quanta. If the edge has a negative Phase Error ($e < 0$), PBS2 will be shortened by a certain number of Time Quanta.

The number of Time Quanta to lengthen or shorten depends on the magnitude of the Phase Error, and is also limited by the Synchronization Jump Width (SJW) value which is programmable.

- When the magnitude of the Phase Error (e) is less than or equal to the SJW, PBS1/PBS2 are lengthened/shortened by the e number of Time Quanta. This has the same effect as Hard Synchronization.
- When the magnitude of the Phase Error is greater than the SJW, PBS1/PBS2 are lengthened/shortened by the SJW number of Time Quanta. This means it may take multiple bits of synchronization before the Phase Error is entirely corrected.

53.3 Architectural Overview

The major functional blocks of the TWAI controller are shown in Figure 53.3-1.

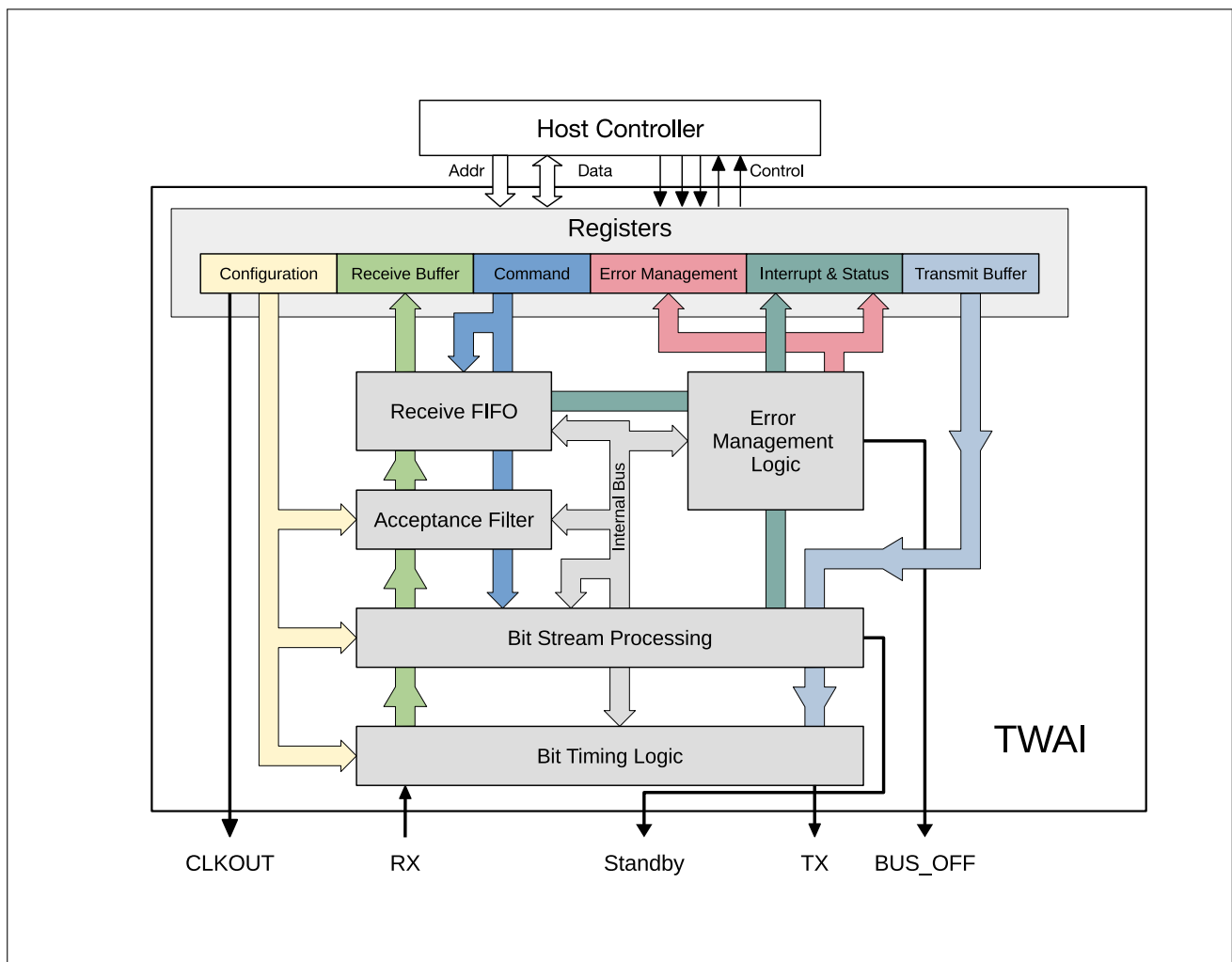


Figure 53.3-1. TWAI Overview Diagram

53.3.1 Bit Timing Logic

The Bit Timing Logic (BTL) module transmits and receives messages at the configured bit rate. The BTL module also handles bit timing synchronization so that communication remains stable. A single bit rate consists of multiple programmable segments that allows users to set the number of time quanta to adjust propagation delay and controller processing time, etc.

53.3.2 Bit Stream Processor

The Bit Stream Processing (BSP) module handles the frame processing of data from the Transmit Buffer (e.g., bit stuffing and additional CRC fields) and generates a bit stream for the Bit Timing Logic (BTL) module. At the same time, the BSP module is also responsible for processing the received bit stream (e.g., de-stuffing and verifying CRC) from the BTL module and placing the message into the Receive FIFO. The BSP will also detect errors on the TWAI bus and report them to the Error Management Logic (EML).

53.3.3 Acceptance Filter

The Acceptance Filter is a programmable message filtering unit that allows the TWAI controller to accept or reject a received message based on the message's ID field. Only accepted messages will be stored in the Receive FIFO. The Acceptance Filter can be configured to Single-filter mode or Dual-filter mode through registers.

53.3.4 Receive FIFO

The Receive FIFO is a 64-byte buffer (inside the TWAI controller) that stores received messages accepted by the Acceptance Filter. Messages in the Receive FIFO can vary in size (between 3 to 13 bytes). When the Receive FIFO is full (or does not have enough space to store the currently received message in its entirety), the Overrun Interrupt will be triggered, and any subsequently received messages will be lost until adequate space is cleared in the Receive FIFO. The first message in the Receive FIFO will be mapped to the 13-byte Receive Buffer until that message is cleared (using the Release Receive Buffer command bit). After being cleared, the Receive Buffer will map to the next message in the Receive FIFO, and the space occupied by the previous message in the Receive FIFO can be used to receive new messages.

53.3.5 Error Management Logic

The Error Management Logic (EML) module updates the TEC and REC, records error information like error types and positions, and updates the error state of the TWAI controller to ensure that the BSP module generates the correct Error Flags. Furthermore, this module also records the bit position when the TWAI controller loses arbitration.

53.3.6 Registers Block

The ESP32-P4 CPU accesses peripherals using 32-bit aligned words. However, the majority of registers in the TWAI controller only contain useful data at the least significant byte (bits [7:0]). Therefore, in these registers, bits [31:8] are ignored on writes, and return 0 on reads.

Configuration Registers

The configuration registers store various configuration items for the TWAI controller such as bit rates, Operation mode, Acceptance Filter, etc. Configuration registers can only be modified whilst the TWAI controller is in Reset mode (See Section [53.4.1](#)).

Command Registers

The command register is used by the CPU to drive the TWAI controller to initiate certain actions such as transmitting a message or clearing the Receive Buffer. The command register can only be modified when the TWAI controller is in Operation mode (see Section [53.4.1](#)).

Interrupt & Status Registers

The interrupt register indicates what events have occurred in the TWAI controller (each event is represented by a separate bit). The status register indicates the current status of the TWAI controller.

Error Management Registers

The error management registers include error counters and capture registers. The error counter registers represent TEC and REC values. The capture registers will record information about instances where the TWAI controller detects a bus error, or when it loses arbitration.

Transmit Buffer Registers

The transmit buffer is a 13-byte buffer used to store a TWAI message to be transmitted.

Receive Buffer Registers

The Receive Buffer is a 13-byte buffer which stores a single message. The Receive Buffer acts as a window of Receive FIFO, whose first message will be mapped into the Receive Buffer.

Note that the Transmit Buffer registers, Receive Buffer registers, and the Acceptance Filter registers share the same address range (offset 0x0040 to 0x0070). Their access is governed by the following rules:

- When the TWAI controller is in Reset mode, all reads and writes to the address range maps to the Acceptance Filter registers.
- When the TWAI controller is in Operation mode:
 - All reads to the address range maps to the Receive Buffer registers.
 - All writes to the address range maps to the Transmit Buffer registers.

53.4 Functional Description

53.4.1 Modes

The ESP32-P4 TWAI controller has two working modes: Reset mode and Operation mode. Reset mode and Operation mode are entered by setting or clearing the [TWAI_RESET_MODE](#) bit.

53.4.1.1 Reset Mode

Entering Reset mode is required in order to modify the various configuration registers of the TWAI controller. When entering Reset mode, the TWAI controller is essentially disconnected from the TWAI bus. When in Reset mode, the TWAI controller will not be able to transmit any messages (including error signals). Any transmission in progress is immediately terminated. Likewise, the TWAI controller will not be able to receive any messages either.

53.4.1.2 Operation Mode

In Operation mode, the TWAI controller connects to the bus and write-protects all configuration registers to ensure consistency during operation. When in Operation mode, the TWAI controller can transmit and receive messages (including error signaling) depending on which operation sub-mode the TWAI controller was configured with. The TWAI controller supports the following operation sub-modes:

- **Normal mode:** The TWAI controller can transmit and receive messages including error signals (such as Error and Overload Frames).
- **Self-test mode:** Self-test mode is similar to Normal mode, but the TWAI controller will consider the transmission of a data or remote frame successful and do not generate an ACK error even if it was not acknowledged. This mode is commonly used during the self-test of a TWAI controller.
- **Listen-only mode:** The TWAI controller will be able to receive messages, but will remain completely passive on the TWAI bus. Thus, the TWAI controller will not be able to transmit any messages, acknowledgments, or error signals. The error counters will remain frozen. This mode is useful for TWAI bus monitoring.

Note that when exiting Reset mode (i.e., entering Operation mode), the TWAI controller must wait for 11 consecutive recessive bits to occur before fully connecting to the TWAI bus (i.e., being able to transmit or receive).

53.4.2 Bit Timing

The operating bit rate of the TWAI controller must be configured whilst the TWAI controller is in Reset mode. The bit rate is configured using [TWAI_BUS_TIMING_0_REG](#) and [TWAI_BUS_TIMING_1_REG](#).

The following Table 53.4-1 illustrates the bit fields of [TWAI_BUS_TIMING_0_REG](#). The frequency of the TWAI core clock has multiple clock sources that can be configured by the user as needed. See Chapter 10 [Reset and Clock](#) for detailed configuration instructions.

Table 53.4-1. Bit Information of [TWAI_BUS_TIMING_0_REG](#) (0x18)

Bit 31-16	Bit 15	Bit 14	Bit 13	Bit 12		Bit 1	Bit 0
Reserved	SJW.1	SJW.0	BRP.13	BRP.12		BRP.1	BRP.0

Notes:

- BRP: The TWAI Time Quanta clock is derived from the XTAL clock (the default is 40 MHz and is configured). The Baud Rate Prescaler (BRP) field is used to define the prescaler according to the equation below, where t_{Tq} is the Time Quanta clock cycle and t_{CLK} is TWAI core clock cycle:

$$t_{Tq} = 2 \times t_{CLK} \times (BRP + 1)$$
- SJW (t_{SJW}): Synchronization Jump Width (SJW) t_{SJW} is configured in SJW[1:0] where $t_{SJW} = (SJW[1:0] + 1)$.

The following Table 53.4-2 illustrates the bit fields of [TWAI_BUS_TIMING_1_REG](#).

Table 53.4-2. Bit Information of `TWAI_BUS_TIMING_1_REG` (0x1c)

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	SAM	PBS2.2	PBS2.1	PBS2.0	PBS1.3	PBS1.2	PBS1.1	PBS1.0

Notes:

- PBS1: The number of Time Quanta in Phase Buffer Segment 1 is defined based on PBS1 + 1.
- PBS2: The number of Time Quanta in Phase Buffer Segment 2 is defined based on PBS2 + 1.
- SAM: Enables triple sampling if set to 1. This is useful for low/medium speed buses to filter spikes on the bus line.

53.4.3 Transmit and Receive Buffers

53.4.3.1 Overview of Buffers

Table 53.4-3. Buffer Layout for Standard Frame Format and Extended Frame Format

Standard Frame Format (SFF)		Extended Frame Format (EFF)	
Offset Address	Content	Offset Address	Content
0x40	TX/RX frame information	0x40	TX/RX frame information
0x44	TX/RX identifier 1	0x44	TX/RX identifier 1
0x48	TX/RX identifier 2	0x48	TX/RX identifier 2
0x4c	TX/RX data byte 1	0x4c	TX/RX identifier 3
0x50	TX/RX data byte 2	0x50	TX/RX identifier 4
0x54	TX/RX data byte 3	0x54	TX/RX data byte 1
0x58	TX/RX data byte 4	0x58	TX/RX data byte 2
0x5c	TX/RX data byte 5	0x5c	TX/RX data byte 3
0x60	TX/RX data byte 6	0x60	TX/RX data byte 4
0x64	TX/RX data byte 7	0x64	TX/RX data byte 5
0x68	TX/RX data byte 8	0x68	TX/RX data byte 6
0x6c	reserved	0x6c	TX/RX data byte 7
0x70	reserved	0x70	TX/RX data byte 8

Table 53.4-3 illustrates the layout of the Transmit Buffer and Receive Buffer registers. Both the Transmit and Receive Buffer registers share the same address space and are only accessible when the TWAI controller is in Operation mode. The CPU accesses Transmit Buffer registers for write operations, and Receive Buffer registers for read operations. Both buffers share the exact same register layout and fields to store a message (received or to be transmitted). The Transmit Buffer registers are used to configure a TWAI message to be transmitted. The CPU would write to the Transmit Buffer registers specifying the message's frame type, frame format, frame ID, and frame data (payload). Once the Transmit Buffer is configured, the CPU would then initiate the transmission by setting the `TWAI_TX_REQUEST` bit in `TWAI_CMD_REG`.

- For a self-reception request, set the `TWAI_SELF_RX_REQUEST` bit instead.
- For a single-shot transmission, set both the `TWAI_TX_REQUEST` and the `TWAI_ABORT_TX` simultaneously.

The Receive Buffer registers map the first message in the Receive FIFO. The CPU would read the Receive Buffer registers to obtain the first message's frame type, frame format, frame ID, and frame data (payload). Once the message has been read from the Receive Buffer registers, the CPU can set the [TWAI_RELEASE_BUFFER](#) bit in [TWAI_CMD_REG](#) to clear the Receive Buffer registers. If there are still messages in the Receive FIFO, the Receive Buffer registers will map the first of the remaining messages again.

53.4.3.2 Frame Information

The frame information is one byte long and specifies a message's frame type, frame format, and length of data. The frame information fields are shown in Table 53.4-4.

Table 53.4-4. TX/RX Frame Information (SFF/EFF); TWAI Address 0x40

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	FF ¹	RTR ²	X ³	X ³	DLC.3 ⁴	DLC.2 ⁴	DLC.1 ⁴	DLC.0 ⁴

Notes:

1. FF: The Frame Format (FF) bit specifies whether the message is Extended Frame Format (EFF) or Standard Frame Format (SFF). The message is EFF when the FF bit is 1, and SFF when the FF bit is 0.
2. RTR: The Remote Transmission Request (RTR) bit specifies whether the message is a data frame or a remote frame. The message is a remote frame when the RTR bit is 1, and a data frame when the RTR bit is 0.
3. X: Don't care, can be any value.
4. DLC: The Data Length Code (DLC) field specifies the number of data bytes for a data frame, or the number of data bytes to request in a remote frame. TWAI data frames are limited to a maximum payload of 8 data bytes, and thus the DLC should range from 0 to 8.

53.4.3.3 Frame Identifier

The Frame Identifier fields occupy two-byte (11-bit) long if the message is SFF, and four-byte (29-bit) long if the message is EFF.

The Frame Identifier fields for an SFF (11-bit) message are shown in Table 53.4-5 ~ 53.4-6.

Table 53.4-5. TX/RX Identifier 1 (SFF); TWAI Address 0x44

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ID.10	ID.9	ID.8	ID.7	ID.6	ID.5	ID.4	ID.3

Table 53.4-6. TX/RX Identifier 2 (SFF); TWAI Address 0x48

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ID.2	ID.1	ID.0	X ¹	X ²	X ²	X ²	X ²

Notes:

1. Don't care. Recommended to be compatible with receive buffer (i.e., set to RTR) in case of using the self-reception functionality (or together with self-test functionality).

2. Don't care. Recommended to be compatible with receive buffer (i.e., set to 0) in case of using the self-reception functionality (or together with self-test functionality).

The Frame Identifier fields for an EFF (29-bits) message is shown in Table 53.4-7 ~ 53.4-10.

Table 53.4-7. TX/RX Identifier 1 (EFF); TWAI Address 0x44

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ID.28	ID.27	ID.26	ID.25	ID.24	ID.23	ID.22	ID.21

Table 53.4-8. TX/RX Identifier 2 (EFF); TWAI Address 0x48

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ID.20	ID.19	ID.18	ID.17	ID.16	ID.15	ID.14	ID.13

Table 53.4-9. TX/RX Identifier 3 (EFF); TWAI Address 0x4c

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ID.12	ID.11	ID.10	ID.9	ID.8	ID.7	ID.6	ID.5

Table 53.4-10. TX/RX Identifier 4 (EFF); TWAI Address 0x50

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ID.4	ID.3	ID.2	ID.1	ID.0	X ¹	X ²	X ²

Notes:

1. Don't care. Recommended to be compatible with receive buffer (i.e., set to RTR) in case of using the self-reception functionality (or together with self-test functionality).
2. Don't care. Recommended to be compatible with receive buffer (i.e., set to 0) in case of using the self-reception functionality (or together with self-test functionality).

53.4.3.4 Frame Data

The Frame Data field contains the payloads of transmitted or received data frame, and can range from 0 to 8 bytes. The number of valid bytes should be equal to the DLC. However, if the DLC is larger than eight bytes, the number of valid bytes would still be limited to eight. Remote frames do not have data payloads, so their Frame Data fields will be unused.

For example, when transmitting a data frame with five bytes, the CPU should write five to the DLC field, and then write data to the corresponding register of the first to the fifth data field. Likewise, when the CPU receives a data frame with a DLC of five data bytes, only the first to the fifth data byte will contain valid payload data for the CPU to read.

53.4.4 Receive FIFO and Data Overruns

The Receive FIFO is a 64-byte internal buffer used to store received messages in First In First Out order. A single received message can occupy between 3 to 13 bytes of space in the Receive FIFO, and their

endianness is identical to the register layout of the Receive Buffer registers. The Receive Buffer registers are mapped to the bytes of the first message in the Receive FIFO.

When the TWAI controller receives the first message, the value of [TWAI_RX_MESSAGE_COUNTER](#) increases to 1, and the RXI interrupt is activated, automatically updating the first message into the buffer. Subsequently, with each new message received by the TWAI controller, the value of [TWAI_RX_MESSAGE_COUNTER](#) increments by 1, reaching a maximum of 64. When this limit is reached and there is sufficient space remaining in the Receive FIFO, the message will be written into the FIFO.

Once the software reads a message from the Receive Buffer, it can free up the space occupied by the message in the Receive FIFO by setting [TWAI_RELEASE_BUFFER](#) to 1. This will also decrement [TWAI_RX_MESSAGE_COUNTER](#) by 1. The Receive Buffer will then map to the next message in the Receive FIFO. The software should repeat the above operations until [TWAI_RX_MESSAGE_COUNTER](#) reaches 0. At this point, the RXI interrupt stops being triggered, the data in the receive buffer becomes invalid, and [TWAI_STATUS_RECEIVE_BUFFER](#) is set to 0.

A data overrun occurs when the TWAI controller receives a message, but the Receive FIFO lacks adequate free space to store the received message in its entirety (either due to the message contents being larger than the free space in the Receive FIFO, or the Receive FIFO being completely full).

When a data overrun occurs:

- The free space left in the Receive FIFO is filled with the partial contents of the overrun message. If the Receive FIFO is already full, then none of the overrun message's contents will be stored.
- When data in the Receive FIFO overruns for the first time, a Data Overrun Interrupt will be triggered.
- Each overrun message will still increment the [TWAI_RX_MESSAGE_COUNTER](#) up to a maximum of 64.
- The Receive FIFO will internally mark overrun messages as invalid. The [TWAI_STATUS_MISS](#) bit can be used to determine whether the message currently mapped to by the Receive Buffer is valid or overrun.

To clear an overrun Receive FIFO, the [TWAI_RELEASE_BUFFER](#) must be called repeatedly until [TWAI_RX_MESSAGE_COUNTER](#) is 0. This requires users to read all valid messages in the Receive FIFO and clear all overrun messages.

53.4.5 Acceptance Filter

The Acceptance Filter allows the TWAI controller to filter out received messages based on their ID (and optionally their first data byte and frame type). Only accepted messages are passed on to the Receive FIFO. The use of Acceptance Filters allows a more lightweight operation of the TWAI controller (e.g., less use of Receive FIFO, fewer Receive Interrupts) since the TWAI Controller only needs to handle a subset of messages.

The Acceptance Filter configuration registers can only be accessed whilst the TWAI controller is in Reset mode, since they share the same address spaces with the Transmit Buffer and Receive Buffer registers.

The configuration registers consist of a 32-bit Acceptance Code Value and a 32-bit Acceptance Mask Value. The Acceptance Code value specifies a bit pattern which each filtered bit of the message must match in order for the message to be accepted. The Acceptance Mask Value is able to mask out certain bits of the Code value (i.e., set as "Don't Care" bits). Each filtered bit of the message must either match the acceptance code or be masked in order for the message to be accepted, as demonstrated in Figure 53.4-1.

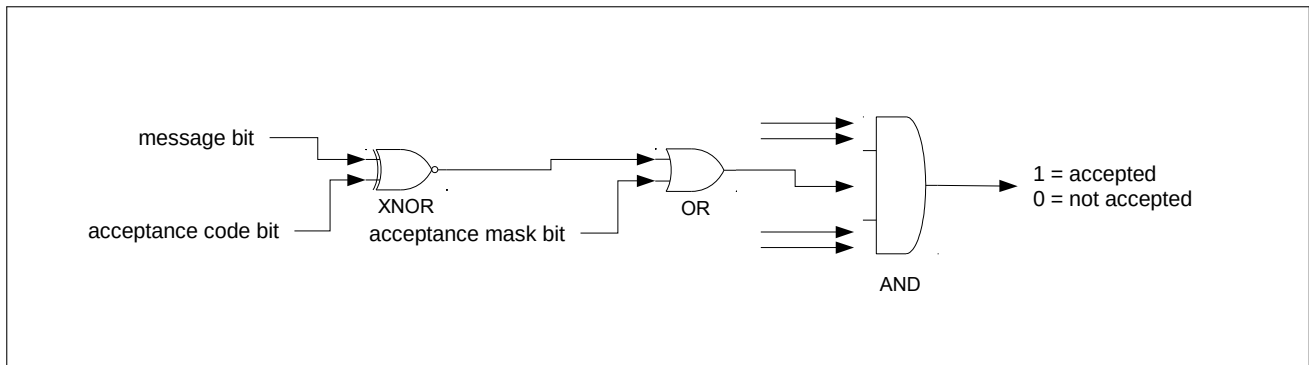


Figure 53.4-1. Acceptance Filter

The TWAI controller Acceptance Filter allows the 32-bit Acceptance Code and Mask Values to either define a single filter (i.e., Single-filter mode), or two filters (i.e., Dual-filter mode). How the Acceptance Filter interprets the 32-bit code and mask values is dependent on the filter mode and the format of received messages (i.e., SFF or EFF).

53.4.5.1 Single-Filter Mode

Single Filter mode is enabled by setting the `TWAI_ACCEPTANCE_FILTER_MODE` bit to 1. This will cause the 32-bit code and mask values to define a single filter. The single filter can filter the following bits of data or remote frames:

- SFF
 - The entire 11-bit ID
 - RTR bit
 - Data byte 1 and Data byte 2
- EFF
 - The entire 29-bit ID
 - RTR bit

The following Figure 53.4-2 illustrates how the 32-bit code and mask values will be interpreted under Single-filter mode.

53.4.5.2 Dual-Filter Mode

Dual-filter mode is enabled by clearing the `TWAI_ACCEPTANCE_FILTER_MODE` bit to 0. This will cause the 32-bit code and mask values to define a two separate filters referred to as filter 1 or filter 2. Under Dual-filter mode, a message will be accepted if it is accepted by one of the two filters.

The two filters can filter the following bits of data or remote frames:

- SFF
 - The entire 11-bit ID
 - RTR bit
 - Data byte 1 (for filter 1 only)

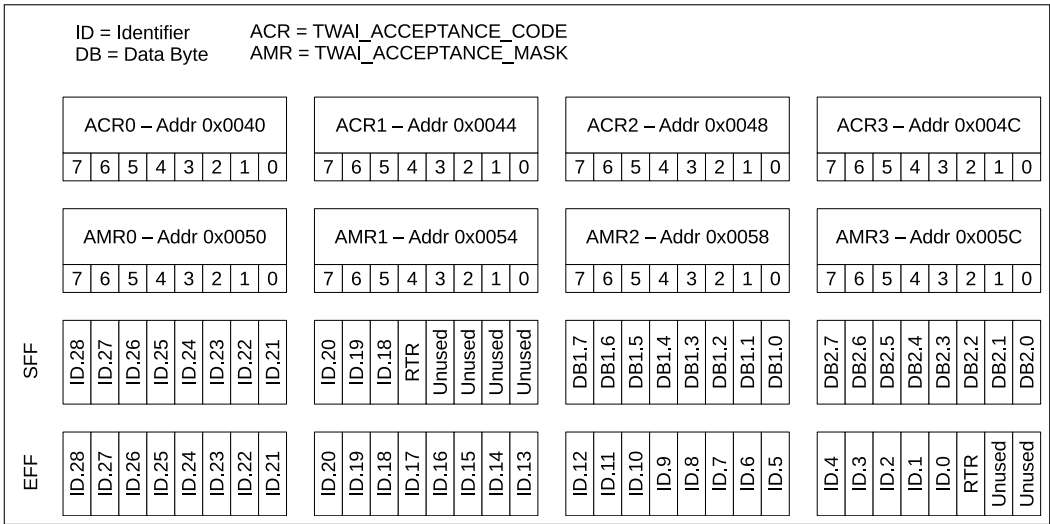


Figure 53.4-2. Single-Filter Mode

- EFF
 - The first 16 bits of the 29-bit ID

The following Figure 53.4-3 illustrates how the 32-bit code and mask values will be interpreted in Dual-filter mode.

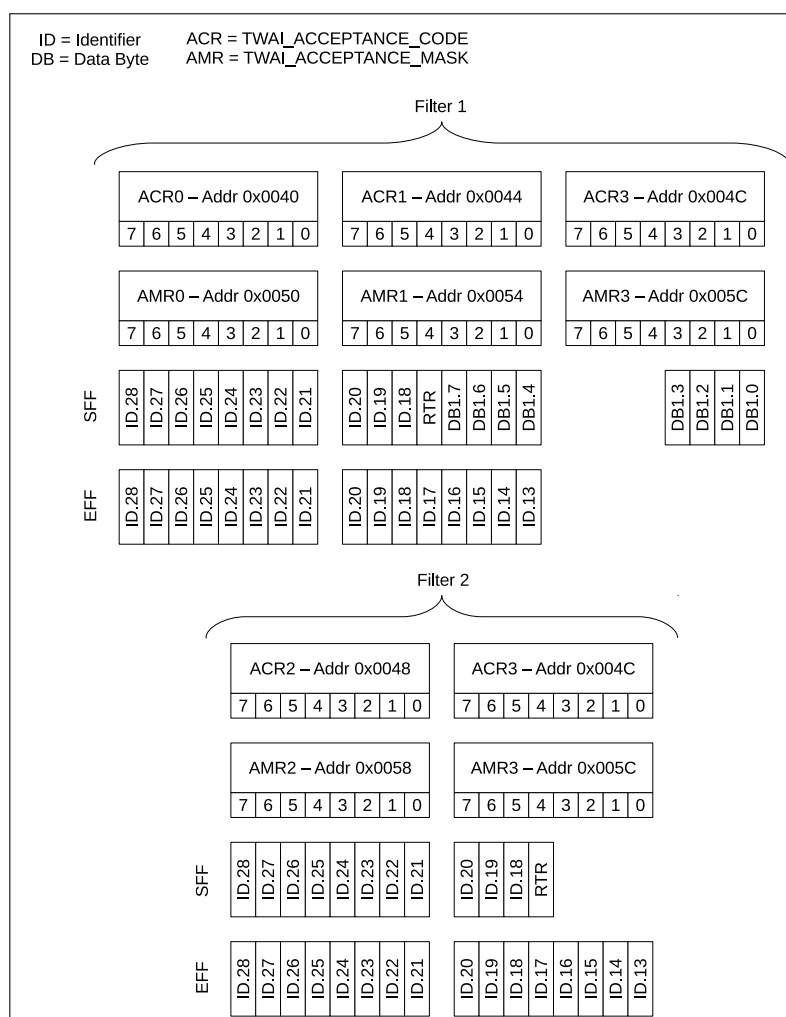


Figure 53.4-3. Dual-Filter Mode

53.4.6 Error Management

The TWAI protocol requires that each TWAI node maintains the Transmit Error Counter (TEC) and Receive Error Counter (REC). The value of both error counters determines the current error state of the TWAI controller (i.e., Error Active, Error Passive, Bus-Off). The TWAI controller stores the TEC and REC values in [TWAI_TX_ERR_CNT_REG](#) and [TWAI_RX_ERR_CNT_REG](#) respectively, and they can be read by the CPU anytime. In addition to the error states, the TWAI controller also offers an Error Warning Limit (EWL) feature that can warn users of the occurrence of severe bus errors before the TWAI controller enters the Error Passive state.

The current error state of the TWAI controller is indicated via a combination of the following values and status bits: TEC, REC, [TWAI_STATUS_ERR](#), and [TWAI_STATUS_NODE_BUS_OFF](#). Certain changes to these values and bits will also trigger interrupts, so that users are notified of error state transitions (see section 53.5). The following figure 53.4-4 shows the relation between the error states, values and bits, and error state related interrupts.

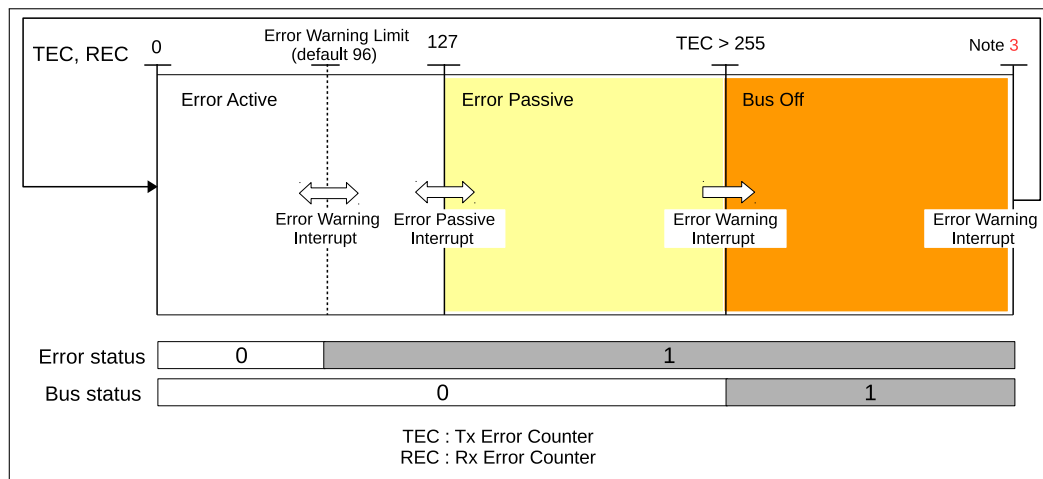


Figure 53.4-4. Error State Transition

53.4.6.1 Error Warning Limit

The Error Warning Limit (EWL) is a configurable threshold value for the TEC and REC, which will trigger an interrupt when exceeded. The EWL is intended to serve as a warning about severe TWAI bus errors, and is triggered before the TWAI controller enters the Error Passive state. The EWL is configured in [TWAI_ERR_WARNING_LIMIT_REG](#) and can only be configured whilst the TWAI controller is in Reset mode. The [TWAI_ERR_WARNING_LIMIT_REG](#) has a default value of 96.

When the values of TEC and/or REC are larger than or equal to the EWL value, the [TWAI_STATUS_ERR](#) bit is immediately set to 1. Likewise, when the values of both the TEC and REC are smaller than the EWL value, the [TWAI_STATUS_ERR](#) bit is immediately reset to 0. The Error Warning Interrupt is triggered whenever the value of the [TWAI_STATUS_ERR](#) bit (or the [TWAI_STATUS_NODE_BUS_OFF](#)) changes.

53.4.6.2 Error Passive

The TWAI controller is in the Error Passive state when the TEC or REC value exceeds 127. Likewise, when both the TEC and REC are less than or equal to 127, the TWAI controller enters the Error Active state. The Error Passive Interrupt is triggered whenever the TWAI controller transitions from the Error Active state to the Error Passive state or vice versa.

53.4.6.3 Bus-Off and Bus-Off Recovery

The TWAI controller enters the Bus-Off state when the TEC value exceeds 255. On entering the Bus-Off state, the TWAI controller will automatically do the following:

- Set REC to 0
- Set TEC to 127
- Set the [TWAI_STATUS_NODE_BUS_OFF](#) bit to 1
- Enter Reset mode

The Error Warning Interrupt is triggered whenever the value of the [TWAI_STATUS_NODE_BUS_OFF](#) bit (or the [TWAI_STATUS_ERR](#) bit) changes.

To return to the Error Active state, the TWAI controller must undergo Bus-Off Recovery. Bus-Off Recovery requires the TWAI controller to observe 128 occurrences of 11 consecutive recessive bits on the bus. To initiate Bus-Off Recovery (after entering the Bus-Off state), the TWAI controller should enter Operation mode by setting the [TWAI_RESET_MODE](#) bit to 0. The TEC tracks the progress of Bus-Off Recovery by decrementing the TEC each time when the TWAI controller observes 11 consecutive recessive bits. When Bus-Off Recovery has completed (i.e., TEC has decremented from 127 to 0), the [TWAI_STATUS_NODE_BUS_OFF](#) bit will automatically be reset to 0, thus triggering the Error Warning Interrupt.

53.4.7 Error Code Capture

The Error Code Capture (ECC) feature allows the TWAI controller to record the error type and bit position of a TWAI bus error in the form of an error code. Upon detecting a TWAI bus error, the Bus Error Interrupt is triggered and the error code is recorded in [TWAI_ERR_CODE_CAP_REG](#). Subsequent bus errors will trigger the Bus Error Interrupt, but their error codes will not be recorded until the current error code is read from [TWAI_ERR_CODE_CAP_REG](#).

The following Table 53.4-11 shows the fields of the [TWAI_ERR_CODE_CAP_REG](#):

Table 53.4-11. Bit Information of [TWAI_ERR_CODE_CAP_REG](#) (0x30)

Bit 31-8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	ERRC.1 ¹	ERRC.0 ¹	DIR ²	SEG.4 ³	SEG.3 ³	SEG.2 ³	SEG.1 ³	SEG.0 ³

Notes:

- **ERRC:** The Error Code (ERRC) indicates the type of bus error; 00 for bit error, 01 for format error, 10 for stuff error, and 11 for other types of error.
- **DIR:** The Direction (DIR) indicates whether the TWAI controller was transmitting or receiving when the bus error occurred; 0 for the transmitter, 1 for the receiver.
- **SEG:** The Error Segment (SEG) indicates the segment of the TWAI message at which the bus error occurred.

The following Table 53.4-12 shows how to interpret the SEG.0 to SEG.4 bits.

Table 53.4-12. Bit Information of Bits SEG.4 - SEG.0

Bit SEG.4	Bit SEG.3	Bit SEG.2	Bit SEG.1	Bit SEG.0	Description
0	0	0	1	1	start of frame
0	0	0	1	0	ID.28 ~ ID.21
0	0	1	1	0	ID.20 ~ ID.18
0	0	1	0	0	bit SRTR
0	0	1	0	1	bit IDE
0	0	1	1	1	ID.17 ~ ID.13
0	1	1	1	1	ID.12 ~ ID.5
0	1	1	1	0	ID.4 ~ ID.0
0	1	1	0	0	bit RTR
0	1	1	0	1	reserved bit 1
0	1	0	0	1	reserved bit 0

Cont'd on next page

Table 53.4-12 – cont'd from previous page

Bit SEG.4	Bit SEG.3	Bit SEG.2	Bit SEG.1	Bit SEG.0	Description
0	1	0	1	1	data length code
0	1	0	1	0	data field
0	1	0	0	0	CRC sequence
1	1	0	0	0	CRC delimiter
1	1	0	0	1	ACK slot
1	1	0	1	1	ACK delimiter
1	1	0	1	0	end of frame
1	0	0	1	0	intermission
1	0	0	0	1	active error flag
1	0	1	1	0	passive error flag
1	0	0	1	1	tolerate dominant bits
1	0	1	1	1	error delimiter
1	1	1	0	0	overload flag

Notes:

- Bit SRTR: under Standard Frame Format.
- Bit IDE: Identifier Extension Bit, 0 for Standard Frame Format.

53.4.8 Arbitration Lost Capture

The Arbitration Lost Capture (ALC) feature allows the TWAI controller to record the bit position where it loses arbitration. When the TWAI controller loses arbitration, the bit position is recorded in [TWAI_ARB_LOST_CAP_REG](#) and the Arbitration Lost Interrupt is triggered.

Subsequent losses in arbitration will trigger the Arbitration Lost Interrupt, but will not be recorded in [TWAI_ARB_LOST_CAP_REG](#) until the current Arbitration Lost Capture is read from the [TWAI_ERR_CODE_CAP_REG](#).

Table 53.4-13 illustrates bits and fields of [TWAI_ERR_CODE_CAP_REG](#) whilst Figure 53.4-5 illustrates the bit positions of a TWAI message.

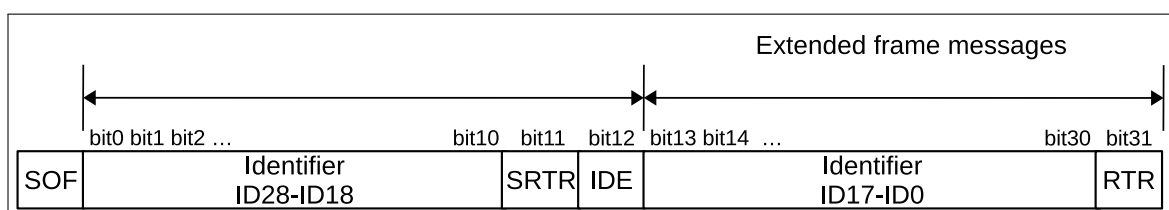


Figure 53.4-5. Positions of Arbitration Lost Bits

Table 53.4-13. Bit Information of [TWAI_ARB_LOST_CAP_REG](#) (0x2c)

Bit 31-5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	BITNO.4 ¹	BITNO.3 ¹	BITNO.2 ¹	BITNO.1 ¹	BITNO.0 ¹

Notes:

- BITNO: Bit Number (BITNO) indicates the nth bit of a TWAI message where arbitration was lost.

53.4.9 Transceiver Auto-Standby

It is common for TWAI transceivers to support a Standby mode to lower power consumption. TWAI transceivers will usually generate a standby signal that is asserted by the connected TWAI controller, thus allowing the controller to place the transceiver into standby when appropriate (e.g., when the bus will be idle for an extended period of time). Transceivers will exit Standby mode if the controller de-asserts the standby signal, or if the transceiver detects bus activity (also known as a wake-up feature).

ESP32-P4's TWAI controller supports both hardware control (i.e., automatic) and software control (i.e., manual) of the standby signal to control the switching of TWAI transceivers connected to the chip. When hardware controlled, the TWAI controller will automatically assert the standby signal when the bus remains idle for longer than a configurable amount of time. When software controlled, the standby signal can be manually asserted/de-asserted directly by the software.

- Hardware output:
 1. Set the [TWAI_HW_STANDBY_EN](#) field in the [TWAI_HW_CFG_REG](#) register to enable standby function for hardware.
 2. Configure the [TWAI_HW_STANDB_CNT_REG](#) register. This register indicates the time required before hardware triggers the standby signal after entering idle status, in which the value indicates the number of cycles of the TWAI controller operating clock (40 MHz by default).
- Software output:
 1. Set the [TWAI_SW_STANDBY_EN](#) field in the [TWAI_SW_STANDBY_CFG_REG](#) register to generate standby signals in the TWAI controller.

The standby signal generated using either of the above methods will be pulled down (cleared) when either of the following conditions is met:

1. The standby signal will be automatically cleared when the TWAI controller's receiver exits the idle status (such as when receiving a new message).
2. Users can also pull down the standby signal by setting the [TWAI_SW_STANDBY_CLR](#) field in the [TWAI_SW_STANDBY_CFG_REG](#) register.

53.5 Interrupts

ESP32-P4's TWAI can generate the following interrupt signal that will be sent to the [Interrupt Matrix](#).

- TWAI_INT

There are several internal interrupt sources from the TWAI that can generate the above interrupt signal. The interrupt sources from the TWAI are listed with their trigger conditions and the resulted interrupt signal in Table 53.5-1.

Table 53.5-1. TWAI's Internal Interrupt Sources

Internal Interrupt Source	Trigger Condition	Interrupt Signal
TWAI_RECEIVE_INT (RXI)	The Receive Interrupt (RXI) is asserted whenever the TWAI controller has received messages that are pending to be read from the Receive Buffer (i.e., when TWAI_RX_MESSAGE_COUNTER_REG > 0). Pending received messages includes valid messages in the Receive FIFO and also overrun messages. The RXI will not be deasserted until all pending received messages are cleared using the TWAI_RELEASE_BUFFER command bit.	TWAI_INT
TWAI_TRANSMIT_INT (TXI)	The Transmit Interrupt (TXI) is triggered whenever Transmit Buffer becomes free, indicating another message can be loaded into the Transmit Buffer for transmission. The Transmit Buffer becomes free under the following scenarios: • A message transmission has completed successfully, i.e., acknowledged without any errors. Any failed messages will automatically be resent. • A single shot transmission has completed (successfully or unsuccessfully, indicated by the TWAI_STATUS_TRANSMISSION_COMPLETE bit). • A message transmission was aborted using the TWAI_ABORT_TX command bit.	TWAI_INT

Cont'd on next page

Table 53.5-1 – cont'd from previous page

Internal Interrupt Source	Trigger Condition	Interrupt Signal
TWAI_ERR_WARNING_INT (EWI)	The Error Warning Interrupt (EWI) is triggered whenever there is a change to the TWAI_STATUS_ERR and TWAI_STATUS_NODE_BUS_OFF bits of the TWAI_STATUS_REG (i.e., transition from 0 to 1 or vice versa). Thus, an EWI could indicate one of the following events, depending on the values TWAI_STATUS_ERR and TWAI_STATUS_NODE_BUS_OFF at the moment when the EWI is triggered. • If TWAI_STATUS_ERR = 0 and TWAI_STATUS_NODE_BUS_OFF = 0: – If the TWAI controller was in the Error Active state, it indicates both the TEC and REC have returned below the threshold value set by TWAI_ERR_WARNING_LIMIT_REG. – If the TWAI controller was previously in the Bus Off Recovery state, it indicates that Bus Recovery has completed successfully. • If TWAI_STATUS_ERR = 1 and TWAI_STATUS_NODE_BUS_OFF = 0: The TEC or REC error counters have exceeded the threshold value set by TWAI_ERR_WARNING_LIMIT_REG. • If TWAI_STATUS_ERR = 1 and TWAI_STATUS_NODE_BUS_OFF = 1: The TWAI controller has entered the BUS_OFF state (due to the TEC >= 256). • If TWAI_STATUS_ERR = 0 and TWAI_STATUS_NODE_BUS_OFF = 1: The TWAI controller's TEC has dropped below the threshold value set by TWAI_ERR_WARNING_LIMIT_REG during BUS_OFF recovery.	TWAI_INT

Cont'd on next page

Table 53.5-1 – cont'd from previous page

Internal Interrupt Source	Trigger Condition	Interrupt Signal
TWAI_DATA_OVERRUN_INT (DOI)	The Data Overrun Interrupt (DOI) is triggered whenever the Receive FIFO has overrun. The DOI indicates that the Receive FIFO is full and should be cleared immediately to prevent any further overrun messages. The DOI is only triggered by the first message that causes the Receive FIFO to overrun (i.e., the transition from the Receive FIFO not being full to the Receive FIFO overflowing). Any subsequent overrun messages will not trigger the DOI again. The DOI could be triggered again when all received messages (valid or overrun) have been cleared.	TWAI_INT
TWAI_TS_COUNTER_OVFL_INT	The Timestamp Overflow Interrupt is triggered whenever the 32-bit counter of the TWAI controller timestamp has overflowed.	TWAI_INT
TWAI_ERR_PASSIVE_INT (EPI)	The Error Passive Interrupt (EPI) is triggered whenever the TWAI controller switches from Error Active to Error Passive, or vice versa.	TWAI_INT
TWAI_ARBITRATION_LOST_INT (ALI)	The Arbitration Lost Interrupt (ALI) is triggered whenever the TWAI controller is attempting to transmit a message and loses arbitration. The bit position where the TWAI controller lost arbitration is automatically recorded in Arbitration Lost Capture register (TWAI_ARB_LOST_CAP_REG). When the ALI occurs again, the Arbitration Lost Capture register will no longer record the new bit location until it is cleared (via CPU reading this register).	TWAI_INT
TWAI_BUS_ERR_INT (BEI)	The Bus Error Interrupt (BEI) is triggered whenever the TWAI controller detects an error on the TWAI bus. When a bus error occurs, the Bus Error type and its bit position are automatically recorded in the Error Code Capture register (TWAI_ERR_CODE_CAP_REG). When the BEI occurs again, the Error Code Capture register will no longer record new error information until it is cleared (via a read from the CPU).	TWAI_INT
TWAI_IDLE_INT (BISI)	The Bus Idle Status Interrupt (BISI) is triggered when the number of clock cycles of the TWAI controller in the idle status exceeds the pre-configured value in the TWAI_IDLE_INTR_CNT_REG register. Users can configure this interrupt to get the TWAI controller idle status and further decide whether to turn off the external TWAI receiver to reduce the overall power consumption (see Section 53.4.9).	TWAI_INT

The TWAI controller's interrupt signal to the interrupt matrix will be asserted whenever one or more interrupt bits are set in the [TWAI_INTERRUPT_REG](#), and de-asserted when all bits in [TWAI_INTERRUPT_REG](#) are cleared. The majority of interrupt bits in [TWAI_INTERRUPT_REG](#) are automatically cleared when the register is read, except for the Receive Interrupt which can only be cleared when all the messages are released by setting the [TWAI_RELEASE_BUFFER](#) bit.

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The specific registers can be found in Section [53.6 Register Summary](#).

53.6 Register Summary

'|' here means a separate line to distinguish between TWAI working modes discussed in Section [53.4.1 Modes](#). The left describes the access in Operation mode. The right belongs to Reset mode and is marked in red. The addresses in this section are relative to Two-wire Automotive Interface base address provided in Table [7.3-2](#) in Chapter [7 System and Memory](#).

The abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

Name	Description	Address	Access
Configuration Registers			
TWAI_MODE_REG	Mode Register	0x0000	R/W
TWAI_CMD_REG	Command Register	0x0004	WO
TWAI_BUS_TIMING_0_REG	Bus Timing Register 0	0x0018	R/W
TWAI_BUS_TIMING_1_REG	Bus Timing Register 1	0x001C	R/W
TWAI_ERR_WARNING_LIMIT_REG	Error Warning Limit Register	0x0034	R/W
TWAI_CLOCK_DIVIDER_REG	Clock Divider Register	0x007C	R/W
TWAI_SW_STANDBY_CFG_REG	Software Standby Register	0x0080	R/W
TWAI_HW_CFG_REG	Hardware Standby Register	0x0084	R/W
TWAI_HW_STANDBY_CNT_REG	Standby Time Length Register	0x0088	R/W
TWAI_IDLE_INTR_CNT_REG	Idle Status Time Length Register	0x008C	R/W
Status Registers			
TWAI_STATUS_REG	Status Register	0x0008	RO
TWAI_ARB_LOST_CAP_REG	Arbitration Lost Capture Register	0x002C	RO
TWAI_ERR_CODE_CAP_REG	Error Code Capture Register	0x0030	RO
TWAI_RX_ERR_CNT_REG	Receive Error Counter Register	0x0038	R/W
TWAI_TX_ERR_CNT_REG	Transmit Error Counter Register	0x003C	R/W
TWAI_RX_MESSAGE_COUNTER_REG	Receive Message Counter Register	0x0074	RO
Interrupt Registers			
TWAI_INTERRUPT_REG	Interrupt Mask Register	0x000C	RO
TWAI_INTERRUPT_ENABLE_REG	Interrupt Enable Register	0x0010	varies
Data Registers			
TWAI_DATA_0_REG	Data Register 0	0x0040	R/W
TWAI_DATA_1_REG	Data Register 1	0x0044	R/W
TWAI_DATA_2_REG	Data Register 2	0x0048	R/W
TWAI_DATA_3_REG	Data Register 3	0x004C	R/W
TWAI_DATA_4_REG	Data Register 4	0x0050	R/W
TWAI_DATA_5_REG	Data Register 5	0x0054	R/W
TWAI_DATA_6_REG	Data Register 6	0x0058	R/W
TWAI_DATA_7_REG	Data Register 7	0x005C	R/W
TWAI_DATA_8_REG	Data Register 8	0x0060	R/W R/O
TWAI_DATA_9_REG	Data Register 9	0x0064	R/W R/O
TWAI_DATA_10_REG	Data Register 10	0x0068	R/W R/O
TWAI_DATA_11_REG	Data Register 11	0x006C	R/W R/O
TWAI_DATA_12_REG	Data Register 12	0x0070	R/W R/O

Name	Description	Address	Access
Timestamp Register			
TWAI_TIMESTAMP_DATA_REG	Timestamp Data Register	0x0094	RO
TWAI_TIMESTAMP_PRESCALER_REG	Timestamp Configuration Register 0	0x0098	R/W
TWAI_TIMESTAMP_CFG_REG	Timestamp Configuration Register 1	0x009C	R/W

53.7 Registers

'|' here means a separate line. The left describes the access in Operation mode. The right belongs to Reset mode with red color. The addresses in this section are relative to Two-wire Automotive Interface base address (each TWAI 0 and TWAI 1 has an individual base address) provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 53.1. TWAI_MODE_REG (0x0000)

(reserved)																												TWAI_ACCEPTANCE_FILTER_MODE				
																												TWAI_SELF_TEST_MODE				
																												TWAI_LISTEN_ONLY_MODE				
																												TWAI_RESET_MODE				
31																												4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	Reset

TWAI_RESET_MODE Configures the Operation mode of the TWAI Controller.

- 0: Operation mode
 - 1: Reset mode
- (R/W)

TWAI_LISTEN_ONLY_MODE Configures whether to enter the Listen-only mode.

- 0: No effect
 - 1: Listen-only mode. In this mode, the nodes will only receive messages from the bus, without generating the acknowledge signal or updating the RX error counter.
- (R/W)

TWAI_SELF_TEST_MODE Configures whether to enter the Self-test mode.

- 0: No effect
 - 1: Enter the Self-test mode. In this mode, the TX nodes can perform a successful transmission without receiving the acknowledge signal. This mode is often used to test a single node with the self-reception request command.
- (R/W)

TWAI_ACCEPTANCE_FILTER_MODE Configures the filter mode.

- 0: Dual-filter mode
 - 1: Single-filter mode
- (R/W)

Register 53.2. TWAI_CMD_REG (0x0004)

(reserved)																												TWAI_SELF_RX_REQUEST TWAI_CLEAR_DATA_OVERRUN TWAI_RELEASE_BUFFER TWAI_ABORT_TX TWAI_TX_REQUEST																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
31																											5	4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

TWAI_TX_REQUEST Configures whether to drive nodes to start transmission.

- 0: No effect
- 1: Drive nodes to start transmission

(WO)

TWAI_ABORT_TX Configures whether to cancel a pending transmission request.

- 0: No effect
- 1: Cancel a pending transmission request

(WO)

TWAI_RELEASE_BUFFER Configures whether to release the RX buffer.

- 0: No effect
- 1: Release the RX buffer

(WO)

TWAI_CLEAR_DATA_OVERRUN Configures whether to clear the data overrun status bit.

- 0: No effect
- 1: Clear the data overrun status bit

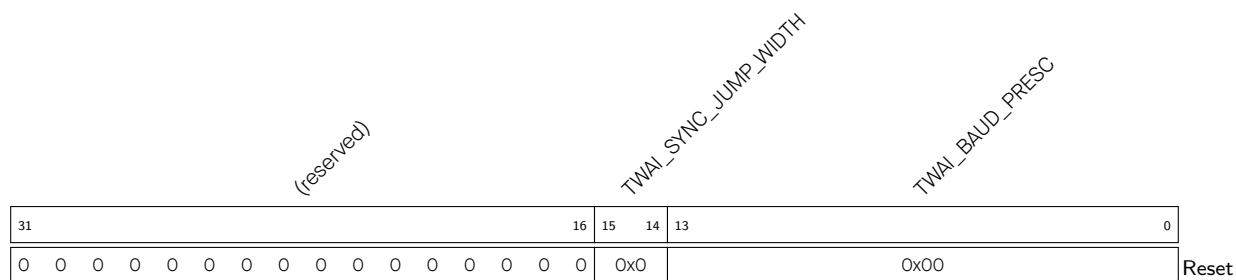
(WO)

TWAI_SELF_RX_REQUEST Configures whether to drive nodes to start transmitting messages while also receiving messages on the bus simultaneously.

- 0: No effect
- 1: Drive nodes to start transmitting messages while also receiving messages on the bus (set the value of [TWAI_TX_REQUEST](#) to 0, or only transmission is allowed)

(WO)

Register 53.3. TWAI_BUS_TIMING_0_REG (0x0018)



TWAI_BAUD_PRESC Configures baud rate prescaler value, determining the frequency dividing ratio.

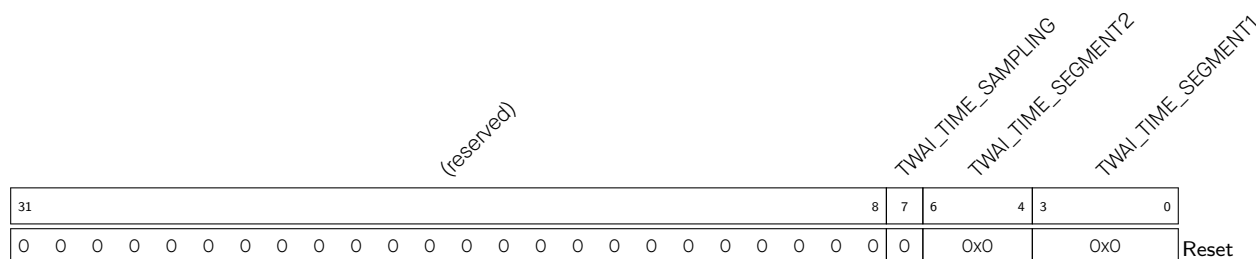
0: Low

1: High

(R/W)

TWAI_SYNC_JUMP_WIDTH Configures Synchronization Jump Width (SJW), ranging from 1 ~ 4 Tq wide. (R/W)

Register 53.4. TWAI_BUS_TIMING_1_REG (0x001C)



TWAI_TIME_SEGMENT1 Configures the width of PBS1. (R/W)

TWAI_TIME_SEGMENT2 Configures the width of PBS2. (R/W)

TWAI_TIME_SAMPLING Configures the number of sample points.

0: The bus is sampled once

1: The bus is sampled three times

(R/W)

Register 53.5. TWAI_ERR_WARNING_LIMIT_REG (0x0034)

(reserved)																								TWAI_ERR_WARNING_LIMIT																	
31																								8	7	0															
0 0																								96																Reset	

TWAI_ERR_WARNING_LIMIT Configures error warning threshold. When any of the error counter values ([TWAI_RX_ERR_CNT](#) or [TWAI_TX_ERR_CNT](#)) exceeds the threshold, the TWAI controller enters an error state; when all the error counter values are below the threshold, the TWAI controller exits the error state. An error warning interrupt will be triggered if the TWAI controller enters or exits an error state. (R/W)

Register 53.6. TWAI_CLOCK_DIVIDER_REG (0x007C)

(reserved)																								TWAI_CLOCK_OFF								TWAI_CD							
31																								9	8	7	0												
0 0																								0	0x0								Reset						

TWAI_CD Configures the divisor of the external CLKOUT pin. (R/W)

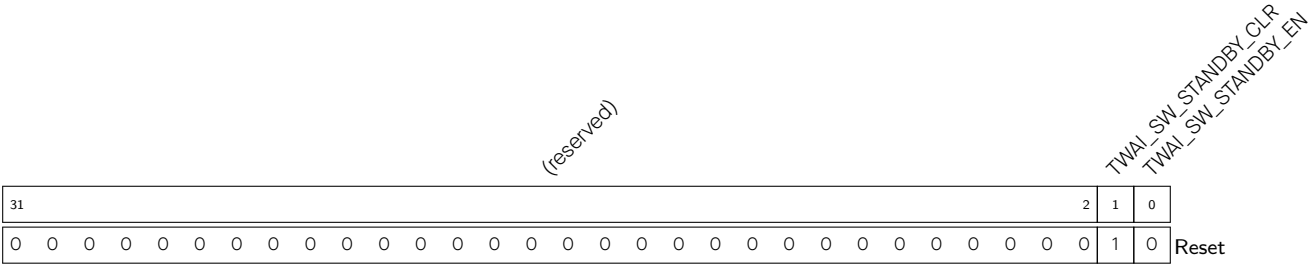
TWAI_CLOCK_OFF Configures whether or not to enable the external CLKOUT pin in Reset mode.

0: Enable the external CLKOUT pin

1: Disable the external CLKOUT pin

(R/W)

Register 53.7. TWAI_SW_STANDBY_CFG_REG (0x0080)



TWAI_SW_STANDBY_EN Configures whether to set standby signals with software.

0: No effect

1: Set standby signals

(R/W)

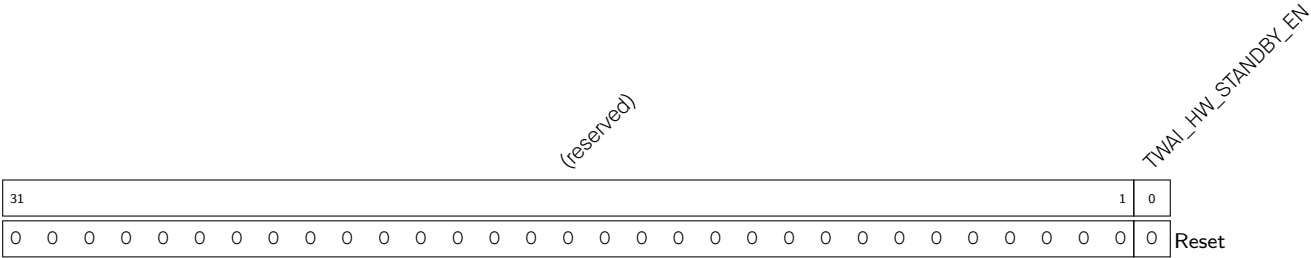
TWAI_SW_STANDBY_CLR Configures whether to clear standby signals with software.

0: No effect

1: Clear standby signals

(R/W)

Register 53.8. TWAI_HW_CFG_REG (0x0084)

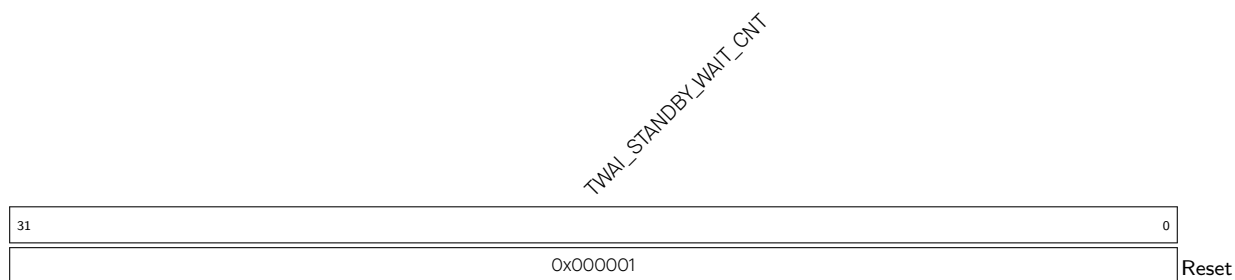


TWAI_HW_STANDBY_EN Configures whether to enable the automatic standby function for hardware.

0: No effect

1: Enable the automatic standby function for hardware

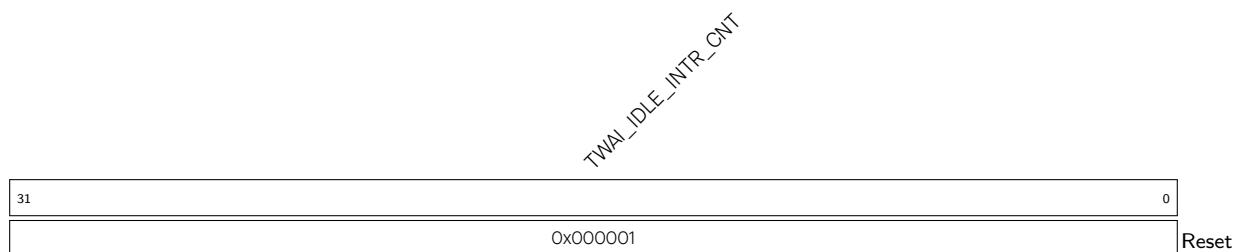
(R/W)

Register 53.9. TWAI_HW_STANDBY_CNT_REG (0x0088)

TWAI_STANDBY_WAIT_CNT Configures the time required before hardware triggers the standby signal after entering idle status.

Measurement unit: TWAI controller clock cycles.

(R/W)

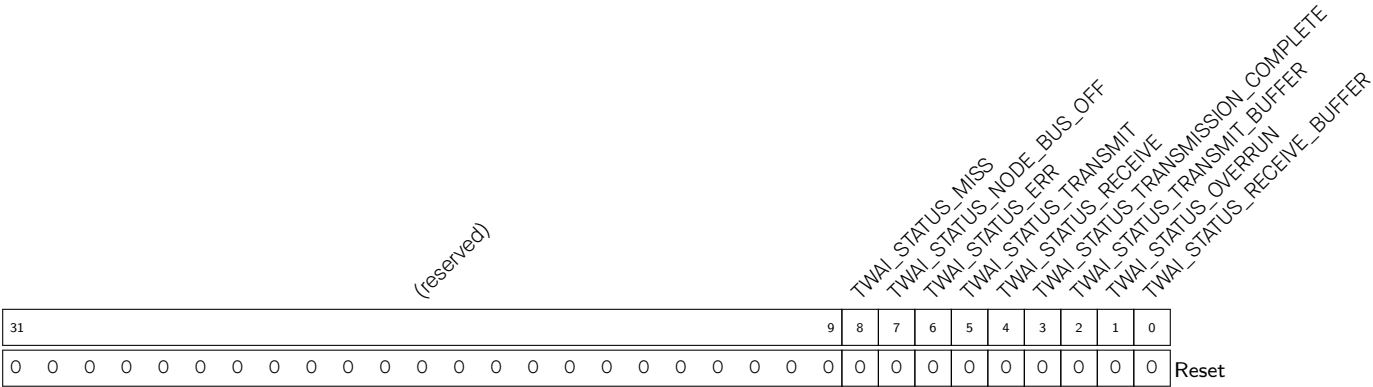
Register 53.10. TWAI_IDLE_INTR_CNT_REG (0x008C)

TWAI_IDLE_INTR_CNT Configures the time required before hardware generates the bus idle status interrupt signal after entering idle status.

Measurement unit: TWAI controller clock cycles.

(R/W)

Register 53.11. TWAI_STATUS_REG (0x0008)



TWAI_STATUS_RECEIVE_BUFFER Represents whether or not the RX buffer is empty.
0: Empty
1: Not empty, with at least one received data packet.
(RO)

TWAI_STATUS_OVERRUN Represents whether or not the RX FIFO is full.
0: Not full
1: Full, and data overrun has occurred
(RO)

TWAI_STATUS_TRANSMIT_BUFFER Represents whether or not the TX buffer is empty.
0: Not empty
1: Empty, and the CPU may write a message into it
(RO)

TWAI_STATUS_TRANSMISSION_COMPLETE Represents whether or not the TWAI controller has sent an entire packet to the bus.
0: Not sent
1: Sent
(RO)

TWAI_STATUS_RECEIVE Represents whether or not the TWAI Controller is receiving a message from the bus.
0: Not receiving
1: Receiving
(RO)

TWAI_STATUS_TRANSMIT Represents whether or not the TWAI Controller is transmitting a message to the bus.
0: Not transmitting
1: Transmitting
(RO)

TWAI_STATUS_ERR Represents at least one of the RX/TX error counter has reached or exceeded the value set in register [TWAI_ERR_WARNING_LIMIT_REG](#). (RO)

Continued on the next page...

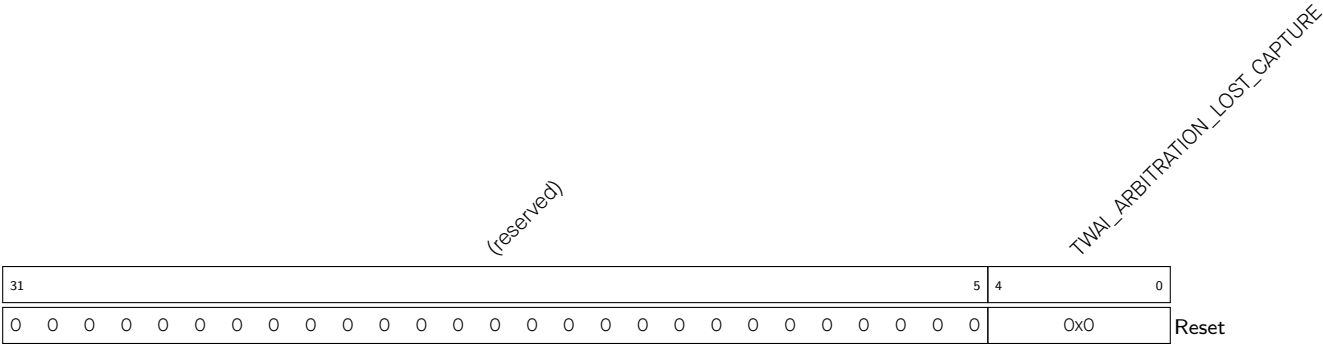
Register 53.11. LP_UART_INT_RAW_REG (0x0004)

Continued from the previous page...

TWAI_STATUS_NODE_BUS_OFF Represents whether or not the TWAI Controller involves in bus activities in bus-off status.
0: Involved
1: No longer involved
(RO)

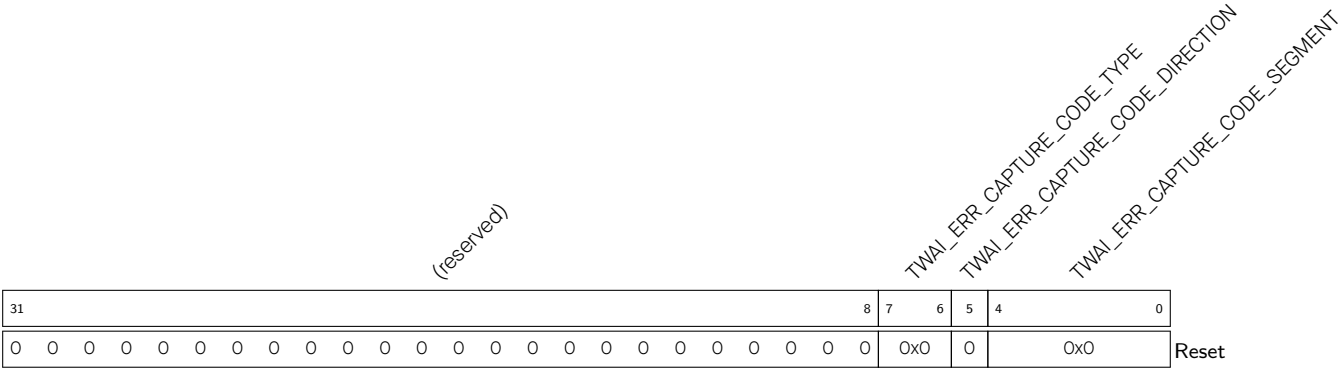
TWAI_STATUS_MISS Represents whether or not the data packet in the RX FIFO is complete.
0: The current packet is complete
1: The current packet is missing
(RO)

Register 53.12. TWAI_ARB_LOST_CAP_REG (0x002C)



TWAI_ARB_LOST_CAPTURE Represents the bit position of lost arbitration. (RO)

Register 53.13. TWAI_ERR_CODE_CAP_REG (0x0030)

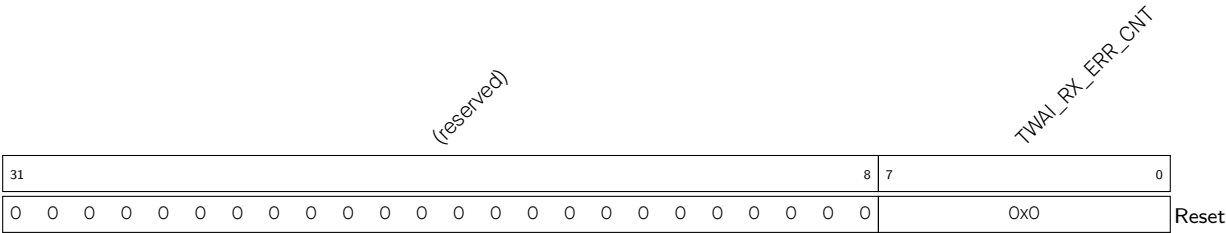


TWAI_ERR_CAPTURE_CODE_SEGMENT Represents the location of errors, see Table 53.4-11 for details. (RO)

TWAI_ERR_CAPTURE_CODE_DIRECTION Represents transmission direction of the node when an error occurs.
0: Error occurs when transmitting a message
1: Error occurs when receiving a message
(RO)

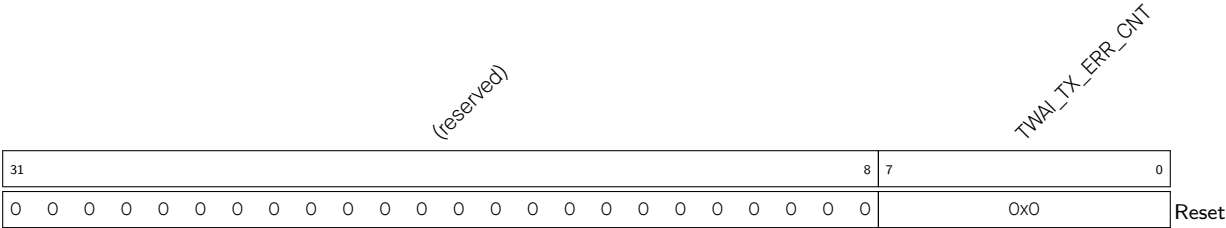
TWAI_ERR_CAPTURE_CODE_TYPE Represents error types.
0: Bit error
1: Form error
2: Stuff error
3: Others
(RO)

Register 53.14. TWAI_RX_ERR_CNT_REG (0x0038)



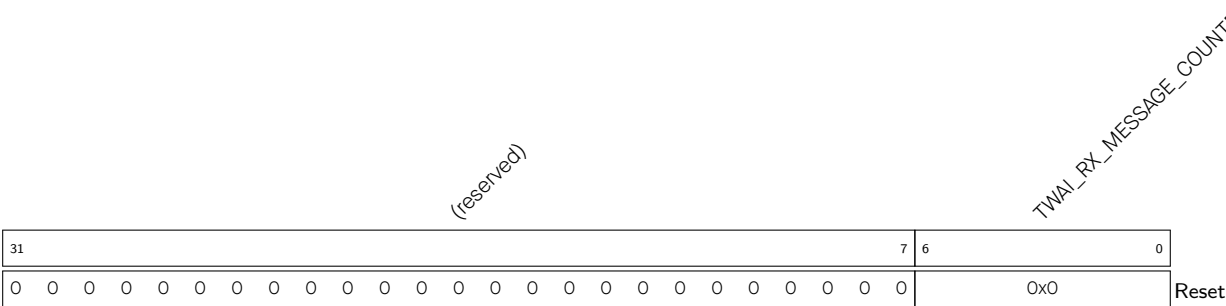
TWAI_RX_ERR_CNT The RX error counter register, which reflects value changes in reception status.
(R/W)

Register 53.15. TWAI_TX_ERR_CNT_REG (0x003C)



TWAI_TX_ERR_CNT The TX error counter register, which reflects value changes in transmission status. (R/W)

Register 53.16. TWAI_RX_MESSAGE_COUNTER_REG (0x0074)



TWAI_RX_MESSAGE_COUNTER Represents the number of messages available within the RX FIFO. (RO)

Register 53.17. TWAI_INTERRUPT_REG (0x000C)

(reserved)																								TWAI_IDLE_INT_ST TWAI_BUS_ERR_INT_ST TWAI_ARBITRATION_LOST_INT_ST TWAI_ERR_PASSIVE_INT_ST TWAI_TS_COUNTER_OVFL_INT_ST TWAI_DATA_OVERRUN_INT_ST TWAI_ERR_WARNING_INT_ST TWAI_TRANSMIT_INT_ST TWAI_RECEIVE_INT_ST											
31																								9	8	7	6	5	4	3	2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset									

TWAI_RECEIVE_INT_ST The masked interrupt status of [TWAI_RX_INT](#). (RO)

TWAI_TRANSMIT_INT_ST The masked interrupt status of [TWAI_TX_INT](#). (RO)

TWAI_ERR_WARNING_INT_ST The masked interrupt status of [TWAI_ERR_WARN_INT](#). (RO)

TWAI_DATA_OVERRUN_INT_ST The masked interrupt status of [TWAI_OVERRUN_INT](#). (RO)

TWAI_TS_COUNTER_OVFL_INT_ST The masked interrupt status of [TWAI_TS_COUNTER_OVFL_INT](#). (RO)

TWAI_ERR_PASSIVE_INT_ST The masked interrupt status of [TWAI_ERR_PASSIVE_INT](#). (RO)

TWAI_ARBITRATION_LOST_INT_ST The masked interrupt status of [TWAI_ARB_LOST_INT](#). (RO)

TWAI_BUS_ERR_INT_ST The masked interrupt status of [TWAI_BUS_ERR_INT](#). (RO)

TWAI_IDLE_INT_ST The masked interrupt status of [TWAI_BUS_STATE_INT](#). (RO)

Register 53.18. TWAI_INTERRUPT_ENABLE_REG (0x0010)

(reserved)																						TWAI_IDLE_INT_ENA TWAI_BUS_ERR_INT_ENA TWAI_ARBITRATION_LOST_INT_ENA TWAI_ERR_PASSIVE_INT_ENA TWAI_TS_COUNTER_OVFL_INT_ENA TWAI_EXT_DATA_OVERRUN_INT_ENA TWAI_EXT_ERR_WARNING_INT_ENA TWAI_EXT_TRANSMIT_INT_ENA TWAI_EXT_RECEIVE_INT_ENA										
31																						9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0											

TWAI_EXT_RECEIVE_INT_ENA Write 1 to enable the [TWAI_RX_INT](#) interrupt. (R/W)

TWAI_EXT_TRANSMIT_INT_ENA Write 1 to enable the [TWAI_TX_INT](#) interrupt. (R/W)

TWAI_EXT_ERR_WARNING_INT_ENA Write 1 to enable the [TWAI_ERR_WARN_INT](#) interrupt. (R/W)

TWAI_EXT_DATA_OVERRUN_INT_ENA Write 1 to enable the [TWAI_OVERRUN_INT](#) interrupt. (R/W)

TWAI_TS_COUNTER_OVFL_INT_ENA Write 1 to enable the [TWAI_TS_COUNTER_OVFL_INT](#) interrupt. (R/W)

TWAI_ERR_PASSIVE_INT_ENA Write 1 to enable the [TWAI_ERR_PASSIVE_INT](#) interrupt. (R/W)

TWAI_ARBITRATION_LOST_INT_ENA Write 1 to enable the [TWAI_ARB_LOST_INT](#) interrupt. (R/W)

TWAI_BUS_ERR_INT_ENA Write 1 to enable the [TWAI_BUS_ERR_INT](#) interrupt. (R/W)

TWAI_IDLE_INT_ENA Write 1 to enable the [TWAI_BUS_STATE_INT](#) interrupt. (RO)

Register 53.19. TWAI_DATA_O_REG (0x0040)

(reserved)																								TWAI_DATA_O									
31																								8	7	0							
0 0																								0x0								Reset	

TWAI_DATA_O In Operation mode, configures the 0th byte information of the data to be transmitted, or reads the 0th byte information of the data received. In Reset mode, configures the 1th byte of the filter code. (R/W)

Register 53.20. TWAI_DATA_1_REG (0x0044)

(reserved)																										TWAI_DATA_1							
31																										8	7	0					
0 0																										0x0						Reset	

TWAI_DATA_1 In Operation mode, configures the 1st byte information of the data to be transmitted, or reads the 1st byte information of the data received. In Reset mode, configures the 2nd byte of the filter code. (R/W)

Register 53.21. TWAI_DATA_2_REG (0x0048)

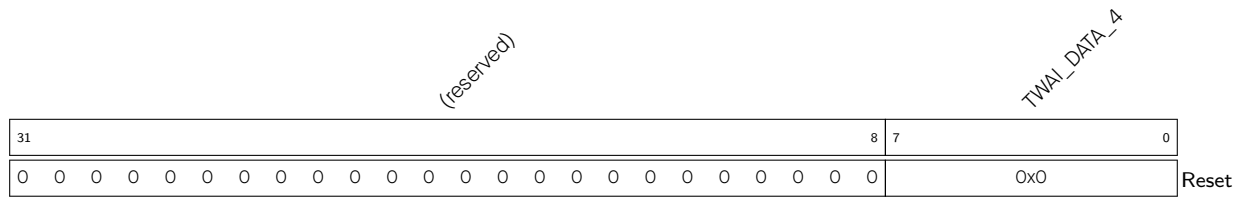
(reserved)																												TWAI_DATA_2								
31																												7	0							
0 0																												0x0								Reset

TWAI_DATA_2 In Operation mode, configures the 2nd byte information of the data to be transmitted, or reads the 2nd byte information of the data received. In Reset mode, configures the 3rd byte of the filter code. (R/W)

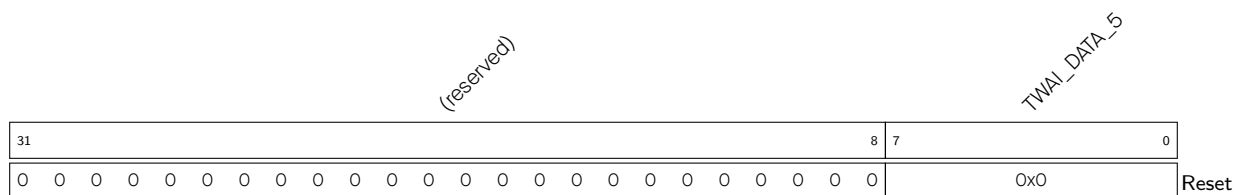
Register 53.22. TWAI_DATA_3_REG (0x004C)

(reserved)																												TWAI_DATA_3									
31																												8	7	0							
0 0																												0x0								Reset	

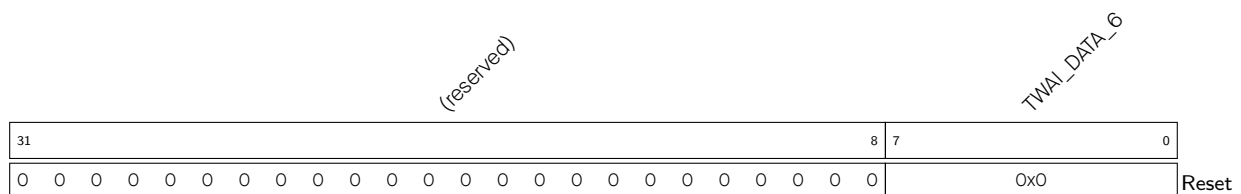
TWAI_DATA_3 In Operation mode, configures the 3rd byte information of the data to be transmitted, or reads the 3rd byte information of the data received. In Reset mode, configures the 4th byte of the filter code. (R/W)

Register 53.23. TWAI_DATA_4_REG (0x0050)

TWAI_DATA_4 In Operation mode, configures the 4th byte information of the data to be transmitted, or reads the 4th byte information of the data received. In Reset mode, configures the 1st byte of the masked filter code. (R/W)

Register 53.24. TWAI_DATA_5_REG (0x0054)

TWAI_DATA_5 In Operation mode, configures the 5th byte information of the data to be transmitted, or reads the 5th byte information of the data received. In Reset mode, configures the 2nd byte of the masked filter code. (R/W)

Register 53.25. TWAI_DATA_6_REG (0x0058)

TWAI_DATA_6 In Operation mode, configures the 6th byte information of the data to be transmitted, or reads the 6th byte information of the data received. In Reset mode, configures the 3rd byte of the masked filter code. (R/W)

Register 53.26. TWAI_DATA_7_REG (0x005C)

(reserved)																TWAI_DATA_7									
31																8	7	0							
0 0																0x0								Reset	

TWAI_DATA_7 In Operation mode, configures the 7th byte information of the data to be transmitted, or reads the 7th byte information of the data received. In Reset mode, configures the 4th byte of the masked filter code. (R/W)

Register 53.27. TWAI_DATA_8_REG (0x0060)

(reserved)																TWAI_DATA_8									
31																8	7	0							
0 0																0x0								Reset	

TWAI_DATA_8 In Operation mode, configures the 8th byte information of the data to be transmitted, or reads the 8th byte information of the data received. (R/W)
In Reset mode, reserved. (RO)

Register 53.28. TWAI_DATA_9_REG (0x0064)

(reserved)																								TWAI_DATA_9									
31																								8	7	0							
0 0																								0x0								Reset	

TWAI_DATA_9 In Operation mode, configures the 9th byte information of the data to be transmitted, or reads the 9th byte information of the data received. (R/W)
In Reset mode, reserved. (RO)

Register 53.29. TWAI_DATA_10_REG (0x0068)

(reserved)																TWAI_DATA_10																	
31																8	7	0															
0 0																0x0																Reset	

TWAI_DATA_10 In Operation mode, configures the 10th byte information of the data to be transmitted, or reads the 10th byte information of the data received. (R/W)
 In Reset mode, reserved. (RO)

Register 53.30. TWAI_DATA_11_REG (0x006C)

(reserved)																TWAI_DATA_11																															
31																7																0															
0 0																0x0																Reset															

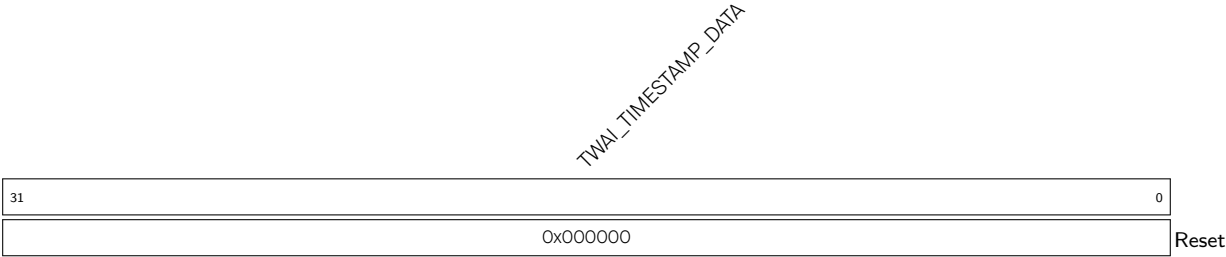
TWAI_DATA_11 In Operation mode, configures the 11th byte information of the data to be transmitted, or reads the 11th byte information of the data received. (R/W)
 In Reset mode, reserved. (RO)

Register 53.31. TWAI_DATA_12_REG (0x0070)

(reserved)																TWAI_DATA_12																	
31																8	7	0															
0 0																0x0																Reset	

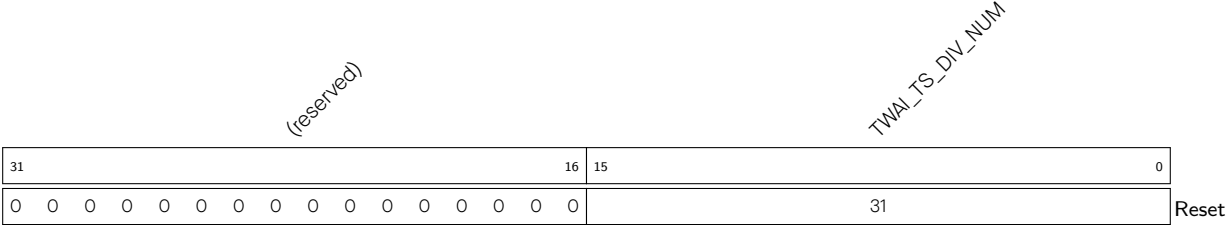
TWAI_DATA_12 In Operation mode, configures the 12th byte information of the data to be transmitted, or reads the 12th byte information of the data received. (R/W)
 In Reset mode, reserved. (RO)

Register 53.32. TWAI_TIMESTAMP_DATA_REG (0x0094)



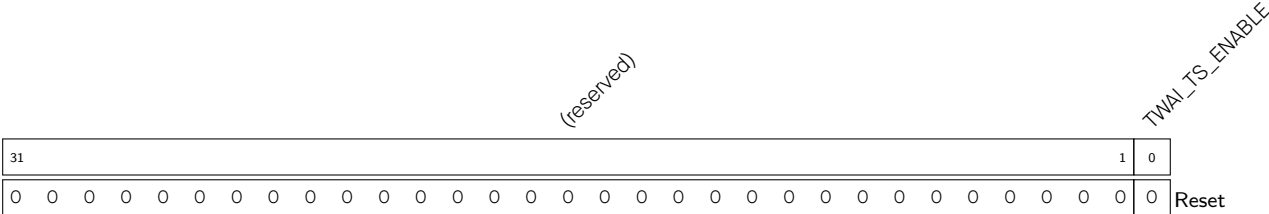
TWAI_TIMESTAMP_DATA The timestamp of the TWAI frame indicated by the RX FIFO read pointer corresponds to the time when the current TWAI frame was received. Valid only when [TWAI_TS_ENABLE](#) is enabled. (RO)

Register 53.33. TWAI_TIMESTAMP_PRESCALER_REG (0x0098)



TWAI_TS_DIV_NUM Configures the clock divisor of the timestamp counter. (R/W)

Register 53.34. TWAI_TIMESTAMP_CFG_REG (0x009C)



TWAI_TS_ENABLE Configures whether to enable the timestamp collection function.

- 0: Disable
- 1: Enable

(R/W)